entropy

MDPI

Article

Comparing Statistical and Machine Learning Methods for Time Series Forecasting in Data-Driven Logistics—A Simulation Study

Lena Schmid [1,*], Moritz Roidl [2], Alice Kirchheim [2,3] and Markus Pauly [1,4]

[1] Department of Statistics, TU Dortmund University, 44227 Dortmund, Germany
[2] Chair of Material Handling and Warehousing, TU Dortmund University, 44227 Dortmund, Germany
[3] Fraunhofer Institute for Material Flow and Logistics, 44227 Dortmund, Germany
[4] Research Center Trustworthy Data Science and Security, University Alliance Ruhr, 44227 Dortmund, Germany
* Correspondence: lena.schmid@tu-dortmund.de

Abstract: Many planning and decision activities in logistics and supply chain management are based on forecasts of multiple time dependent factors. Therefore, the quality of planning depends on the quality of the forecasts. We compare different state-of-the-art forecasting methods in terms of forecasting performance. Differently from most existing research in logistics, we do not perform this in a case-dependent way but consider a broad set of simulated time series to give more general recommendations. We therefore simulate various linear and nonlinear time series that reflect different situations. Our simulation results showed that the machine learning methods, especially Random Forests, performed particularly well in complex scenarios, with the differentiated time series training significantly improving the robustness of the model. In addition, the time series approaches proved to be competitive in low noise scenarios.

Keywords: machine learning; time series; forecasting; simulation study

check for updates

Received: 1 October 2024
Revised: 19 December 2024
Accepted: 25 December 2024
Published: 31 December 2024

Citation: Schmid, L.; Roidl, M.; Kirchheim, A.; Pauly, M. Comparing Statistical and Machine Learning Methods for Time Series Forecasting in Data-Driven Logistics—A Simulation Study. *Entropy* **2025**, *27*, 25. https://doi.org/10.3390/e27010025

Copyright: © 2024 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

1. Introduction

Forecasting methods are essential for efficient planning in various logistics domains such as warehousing, transport, and supply chain management. They enable companies to anticipate and plan for future demand, capacity needs, and supply chain requirements. Thereby, different logistics applications require different forecasts due to their unique characteristics. In the transport domain, for example, accurate transportation forecasting enables logistics companies to optimize their transportation networks, reduce transportation costs, and enhance delivery reliability [1–5]. Precise forecasting allows warehouse managers to optimize space use, reduce stock-out risk, and improve overall efficiency [6,7]. In supply chain management, accurate forecasts are, for example, used to optimize resource use across the entire supply chain [8–10]. The above references show that the use of forecasting techniques such as time series models and machine learning methods has become increasingly popular in logistics in recent years. However, there is still a lack of consensus on which method is more effective, especially as most methods of comparison in logistics solely rely on comparing the performance on a few data sets [7,11]. In fact, differently from other fields (e.g., [12,13]), there do not exist rigorous benchmark studies in data-driven logistics to the best of our knowledge. In our opinion, the key reason for this is that, outside of specific examples (e.g., [14,15]), there is a lack of freely accessible and well-characterized data sets for benchmarking (e.g., [16,17]) in the logistics research domain. This hampers the analysis of domain-specific pros and cons of method choices or the formulation of general recommendations. To overcome this and to be in line with recent recommendations [18],

we therefore focus on simulating data from various statistical time series models that reflect potential logistic scenarios.

Time series models have been used in forecasting for several decades and are widely used in logistics for sales or demand forecasting, see, e.g., [9,19] and the references cited therein. These models are based on historical data and use statistical techniques to identify patterns and trends in the data, which can then be used to make predictions about future demand. Some commonly used time series models in logistics include (seasonal) autoregressive integrated moving averages (ARIMA) and exponential smoothing models. For example, ref. [20] developed an ARIMA multistage supply chain model that is based on time series models. Another example is Prophet [21], a forecasting tool for time series analysis developed by Facebook, which includes additive modeling with components such as seasonality, holidays, and trend flexibility. Ref. [22] examined ARIMA and Prophet models for predicting supermarket sales. The Prophet models showed superior predictive performance in terms of lower errors. Ref. [23] investigated the performance of double exponential smoothing for inventory forecasting.

More recently, machine learning (ML) methods have become increasingly popular for demand forecasting in logistics due to their ability to handle large and complex data sets. There are many literature reviews [24–28] that discuss the use of machine learning techniques in forecasting for supply chain management, including an overview of the various techniques used and their advantages and limitations. However, our comment regarding a lack of neutral benchmarking studies still applies.

Several studies have shown that ML methods such as neural networks, support vector regression, and Random Forests can outperform traditional time series models for specific demand forecasting problems. For example, a study by [11] compared the prediction power of more than ten different forecasting models, including classical methods such as ARIMA and ML techniques such as long short-term memory (LSTM) and convolution neural networks, using a single data set containing the sales history of furniture in a retail store. The results showed that the LSTM outperformed the other models in terms of prediction performance. Another study by [29] also compared the forecasting power of ARIMA and neural networks using a single commodity prices data set. Again the neural network performed better than the ARIMA model. Similar results were obtained in [30,31]. However, other studies have found mixed results, with some suggesting that time series models perform better than ML methods. For instance, ref. [32] compared the forecasting accuracy of ARIMA and neural network models in predicting wind speed for short time intervals. The results showed that the performance of both can be very similar, indicating that a more simple and interpretable forecasting model could be used to administrate energy sources. A comparison of the daily hotel demand forecasting performance of SARIMAX, GARCH, and neural networks also showed that both time series approaches outperformed the neural networks [33]. In the latter examples, one reason may also be the difficulty in tuning complex machine learning procedures. That is one reason why we focus on out-of-the-box machine learning methods in our study.

The comparison of the forecasting performance of ML methods and time series models in logistics has significant implications for businesses seeking to improve their forecasting accuracy. By identifying the most effective forecasting methods, businesses can make better-informed decisions about production, inventory management, and resource allocation. Thus, this work aims to provide a comprehensive comparison of the forecasting performance of time series models and ML methods. Differently from the above-mentioned works that merely focus on single use cases, this task needs more variation in the data sets under study. To this end, we compare various forecasting methods in terms of out-of-the-box forecasting performance on a broad set of simulated time series. We thereby simulate

various linear and nonlinear time series that are of importance for logistics and study the one-step forecast performance of different statistical learning methods.

This work is structured as follows: Section 2 presents the different forecasting methods used. More precisely, the (seasonal) ARIMA and TBATS models are presented. In addition, the machine learning approaches (Random Forest and XGBoost) are described in more detail. Section 3 presents the simulation design and framework, while Section 4 summarizes the main simulation results. In Section 5, an illustrative real-world data example is analyzed before the manuscript concludes with a discussion of our findings and an outlook for future research (Section 6).

2. Methods

In this section, we explain the one-step forecasting methods under investigation. There are various strategies for modeling and forecasting time series. Traditional time series models, including moving averages and exponential smoothing, follow a linear approach in which the predictions of future values are linear functions of past observations. Due to their relative simplicity in terms of understanding and implementation, linear models have found application in many forecasting problems [34–36]. To overcome the limitations of linear models and account for certain nonlinear patterns observed in real-world problems, several classes of nonlinear models have been proposed in the literature. Examples cover the threshold autoregressive model (TAR) [37] or the generalized autoregressive conditional heteroscedastic model (GARCH) [38]. Although some improvements have been noted, the utility of their application to general prediction problems is limited [39]: since these models were developed for specific nonlinear patterns, they are often unable to model other types of nonlinearities. Here, machine learning methods have been proposed as an alternative for time series forecasting [40,41]. Since it is impossible to cover the entire spectrum of machine learning models and time series methods in our simulation study, we limit ourselves to a selection of what we consider the most common algorithms in data-driven logistics. To evaluate the performance, we compare these methods with a naive approach, where the last observation of the time series is used as a prediction. The time series (Section 2.1) and machine learning methods (Section 2.2) under study are explained in more detail in the next two subsections.

2.1. Time Series Methods

We focus on three different time series models: ARIMA, SARIMA, and TBATS. The first two models are among the most popular models in traditional time series forecasting [42,43] and are often used as benchmark models for comparison with machine learning algorithms [44–46]. In addition, TBATS models combine several techniques such as exponential smoothing and Fourier terms, making it particularly adept at handling complex patterns, including multiple seasonalities and nonlinear behaviors [43]. This combination of traditional and advanced methods ensures that we cover a range of forecasting techniques commonly applied in the data-driven logistics domain.

2.1.1. ARIMA

The autoregressive integrated moving average (ARIMA) [47] model is a generalized model of the autoregressive moving average (ARMA) model and builds a composite model of the time series [48]. Denoted as ARIMA(p, d, q), $p, q, d \in \mathbf{N}$, the model is characterized by three key components:

- AR (Autoregression): Represents the regression of the time series on its own past values, capturing dependencies through lagged observations. The number of lagged observations included in the models is given by p.

- I (Integrated): The differencing order (d) indicates the number of times the time series is differenced to achieve stationarity. This transformation involves subtracting the current observation from its d-th lag, which is crucial for stabilizing the mean and addressing trends.
- MA (Moving Average): Incorporates a moving average model to account for dependencies between observations and the residual errors of the lagged observations (q).

In general, a time series $\{x_t\}_t$ generated from an ARIMA(p, d, q) model has the form:

$$\Phi_p(B)(1-B)^d x_t = \Theta_q(B)\varepsilon_t,$$

where $p, d, q \in \mathbb{N}$ and B is the backshift operator defined as $B : \mathbb{R} \to \mathbb{R}$ with $x_t \mapsto x_{t-1}$. The AR component is described by the polynomial $\Phi_p(B) = 1 - \phi_1 B - \phi_2 B^2 - \ldots - \phi_p B^p$, where $\phi_1, \ldots, \phi_p \in \mathbb{R}$. The MA component is represented by the polynomial $\Theta_q(B) = 1 + \theta_1 B + \theta_2 B^2 + \ldots + \theta_q B^q$, where $\theta_1, \ldots, \theta_q \in \mathbb{R}$. Residual errors at time t, denoted as ε_t, are assumed to follow a white noise process with zero mean and constant variance.

2.1.2. SARIMA

With seasonal time series data, short-term non-seasonal components likely contribute to the model. Therefore, we need to estimate a seasonal ARIMA model incorporating non-seasonal and seasonal factors into a multiplicative model [48]. The general form of a seasonal ARIMA model is denoted as SARIMA$(p, d, q)(P, D, Q)_m$, where p is the non-seasonal AR order, d is the non-seasonal differencing, q is the non-seasonal MA order, and P, D, and Q are the similar parameters for the seasonal part. The parameter mm represents the number of time steps in one full seasonal cycle, also known as the period length.

2.1.3. TBATS

For time series data exhibiting complex and diverse seasonal patterns, TBATS (Trigonometric Seasonal Exponential Smoothing) is a robust modeling approach. Introduced as an extension of exponential smoothing methods, TBATS accounts for different seasonalities through a combination of trigonometric functions and exponential smoothing [49]. The model is particularly effective in handling multiple seasonal cycles, making it suitable for data sets with intricate temporal structures.

The general form of a TBATS model consists of several components as described below:

- T (Trend): Captures the overall trend in the time series using an exponential smoothing mechanism.
- B (Box–Cox Transformation): Applies the Box–Cox transformation [50] to stabilize variance and ensure the homogeneity of variances.
- A (ARIMA Errors): Incorporates ARIMA errors to capture any remaining non-seasonal dependencies.
- S (Seasonal): Utilizes trigonometric functions to model multiple seasonal components, accommodating various seasonal patterns.

2.2. Machine Learning Methods

Machine learning methods are increasingly being used to address time series prediction problems. In fact, there exist too many approaches to consider in a comparison study like ours. We therefore restricted ourselves to a class that has already been successfully used for predictions in the logistics context [1,51–54]: tree-based ensemble learners. We thereby focus on two models, each studied with and without differencing: Random Forest and XGBoost on trees which are briefly introduced below. These methods are particularly well-suited to time series forecasting due to their flexibility in capturing both linear and

nonlinear patterns, as well as their robustness to overfitting and ability to handle large data sets with complex structures.

2.2.1. XGBoost

Gradient boosting is an ensemble machine learning technique often used in classification and regression problems, and is particularly popular in predictive scenarios [55]. As an ensemble technique, gradient boosting combines the results of several weak learners, referred to as base learners, with the aim of building a model that generally performs better than the conventional single machine learning models. Typically, gradient boosting utilizes decision trees as base learners. Like other boosting methods, the core idea of gradient boosting is that, during the learning procedure, new models are built and fitted consecutively and not independently to provide better predictions of the output variable. Thereby, new base learners are constructed with the aim of minimizing a loss function associated with the whole ensemble. Instances that are not predicted correctly in previous steps and score higher errors are correlated with larger weight values so that the model can focus on them and learn from its mistakes.

XGBoost stands for Extreme Gradient Boosting and is a specific implementation of gradient boosting [56]. It incorporates randomization and regularization techniques to reduce overfitting while increasing training speed. Moreover, it computes second-order gradients of the loss function, which provides more information about the gradient's direction, making it easier to minimize the loss function.

In general, the hyperparameters for XGBoost can be divided into two categories [56]: General boosting parameters, including the number of iterations and the learning rate, which controls how much information from a new tree will be used in the boosting step. Second, in base learner dependent parameters. When trees are used as base learners, the additional hyperparameters are used to control the complexity of the individual trees. Examples include limiting the maximum tree depth or specifying a minimum number of samples in each leaf [57]. There also exist other boosting variants [58–60], but we concentrate on XGBoost as it has emerged as one of the key machine learning models for prediction and was also referred to as 'the Queen of Machine Learning' [61] in this context. XGBoost models have also been used for time series forecasting, e.g., [62,63]. For example, in [64], the potential of XGBoost in retail for predicting store sales was investigated while ref. [1] studied this for predicting the travel time of NYC cabs.

2.2.2. Random Forest

A Random Forest [65] is a machine learning method based on building ensembles of decision trees. It was developed to address predictive shortcomings of traditional Classification and Regression Trees (CARTs) [66]. Random Forests consist of a large number of weak decision tree learners, which are grown in parallel to reduce the bias and variance of the model at the same time [65]. For training a Random Forest, bootstrap samples are drawn from the training data set. Each bootstrap sample is then used to grow a(n unpruned) tree. Instead of using all available features in this step, only a small and fixed number of randomly sampled m_{try} features are selected as split candidates. A split is chosen by the CART-split criterion for regression, i.e., by minimizing the sum of squared errors in both child nodes. Instead of the CART-split criterion, many other distances, such as the least absolute deviations of the mean (L1-norm), can also be used. These steps are repeated until B such trees are grown, and new data are predicted by taking the mean of all B tree predictions. The most important hyperparameters for the Random Forest [67] are as follows:

- B is the number of grown trees. Note that this parameter is usually not tuned since it is known that more trees are better.
- The cardinality of the sample of features at every node is m_{try}.
- The minimum number of observations that each terminal node should contain (stopping criteria).

Though there exist other variants of bagged tree-based ensembles [68,69], we concentrate on the Random Forest as it is the best known method that is often seen as the machine learning benchmark procedure, e.g., [70]. In addition, Random Forests have also been frequently used for time series forecasting [1,71]. For example, in [72], a Random Forest approach was used to model real-time delivery time forecasts in online retailing while ref. [73] applied Random Forest to predict product demand for grocery items.

While machine learning methods are quite en vogue, we should not neglect the advantages of time series methods in terms of interpretability. Here, time series approaches enable a clearer understanding of the factors influencing the predictions.

3. Simulation Set-Up

In our simulation study, we compare the one-step forecast prediction performance of the methods described in Section 2. All simulations were conducted in the statistical computing software `R` [74]. We use the `forecast` package [75] for all time series approaches under consideration. For the machine learning methods, we used the `ranger` [67] and `xgboost` [76] packages for Random Forest and XGBoost, respectively. The concrete simulation settings and data generating processes (DGPs) are described below.

3.1. Data Generating Processes

We consider twelve DGPs in total—an autoregressive model (AR), two bilinear models (BLs), two nonlinear autoregressive models (NARs), a nonlinear moving average model (NMA), two sign autoregressive models (SARs), two smooth transition autoregressive models (STARs) and two TAR models. They are summarized in Table 1, where the error terms ε_t are independent and identically distributed with a standard normal distribution.

Similar models have been used to evaluate time series forecasts [45] and are of importance in data-driven logistics. In particular, autoregressive models (AR, NAR1, and NAR2) are well suited to capturing the temporal persistence and trends often observed in historical logistics demand data, such as warehouse throughput or vehicle routing sequences [77]. Bilinear models (BL1 and BL2) reflect the complex interactions within logistics systems. For example, the interaction between past demand and various external factors such as weather conditions, production schedules, or transportation disruptions can have a substantial impact on future demand patterns. Bilinear models have been shown to capture such intricate interactions effectively, making them suitable for complex logistics environments where multiple variables influence each other simultaneously [78]. The nonlinear moving average (NMA) model is apt for situations where interdependencies exist between multiple factors influencing logistics outcomes, such as supply chain delays, inventory dynamics, or market fluctuations. This model accounts for the nonlinear relationships between past error terms, which can be influenced by the aggregation of multiple small factors. Sign autoregressive models (SAR1, SAR2) are useful in logistics systems where certain events, such as strikes, weather events, or sudden demand shifts, cause abrupt directional changes in future demand. These models can capture such threshold effects, providing valuable insights into logistics system resilience and responsiveness. Smooth transition autoregressive models (STAR1, STAR2) are particularly relevant for logistics systems that experience gradual transitions in demand patterns due to external factors like economic cycles, regulatory changes, or long-term supply chain restructuring. These models can help

to predict how demand or supply dynamics may evolve over time as conditions change smoothly. Threshold autoregressive models (TAR1 and TAR2) are well suited to logistics settings where distinct operational regimes exist, such as different levels of demand or supply based on specific conditions like inventory thresholds or transportation capacity limits. By modeling regime-switching behavior, TAR models can provide insights into logistics processes that exhibit different behaviors under different operational conditions. This diverse set of DGPs depicts many aspects of the multi-layered nature of logistics data, which includes persistence, interactions, complicated dependencies, directional influences, smooth transitions, and different regimes. In the absence of comprehensive benchmark problems, this set-up allows us to evaluate the adaptability of forecasting methods in dynamic logistics scenarios.

Table 1. Data generating processes (DGPs) used in the simulation study. The error terms ε_t are i.i.d $\mathcal{N}(0,1)$.

Model Type	Variant(s)	Data Generating Process
Autoregressive	AR	$x_t = 0.5x_{t-1} + 0.45x_{t-2} + \varepsilon_t$
Bilinear	BL 1	$x_t = 0.7x_{t-1} \cdot \varepsilon_{t-2} + \varepsilon_t$
	BL2	$x_t = 0.4x_{t-1} - 0.3x_{t-2} + 0.5x_{t-2} \cdot \varepsilon_{t-1} + \varepsilon_t$
Nonlinear Autoregressive	NAR 1	$x_t = \frac{0.7\lvert x_{t-1}\rvert}{\lvert x_{t-1}\rvert+2} + \varepsilon$
	NAR2	$x_t = \frac{0.7\lvert x_{t-1}\rvert}{\lvert x_{t-1}\rvert+2} + \frac{0.35\lvert x_{t-2}\rvert}{\lvert x_{t-2}\rvert+2} + \varepsilon$
Nonlinear Moving Average	NMA	$x_t = \varepsilon_t - 0.3\varepsilon_{t-1} + 0.2\varepsilon_{t-2} + 0.4\varepsilon_{t-1}\varepsilon_{t-2} - 0.25\varepsilon_{t-2}^2$
Sign Autoregressive	SAR 1	$x_t = \text{sign}(x_{t-1}) + \varepsilon_t,$
	SAR 2	$x_t = \text{sign}(x_{t-1} + x_{t-2}) + \varepsilon_t,$
Smooth Transition Autoregressive	STAR 1	$x_t = 0.8\varepsilon_t - \frac{0.8\varepsilon_{t-1}}{1+\exp(-10x_{t-1})} + \varepsilon_t,$
	STAR 2	$x_t = 0.3x_t + 0.6x_{t-2} + \frac{0.1-0.9x_{t-1}+0.8x_{t-2}}{1+\exp(-10x_{t-1})} + \varepsilon_t,$
Threshold Autoregressive	TAR 1	$x_t = \begin{cases} 0.9x_{t-1} + \varepsilon_t & \text{if } \lvert x_{t-1}\rvert \leq 1 \\ -0.3x_{t-1} - \varepsilon_t & \text{if } \lvert x_{t-1}\rvert > 1 \end{cases}$
	TAR 2	$x_t = \begin{cases} 0.9x_{t-1} + 0.05x_{t-2} + \varepsilon_t & \text{if } \lvert x_{t-1}\rvert \leq 1 \\ -0.3x_{t-1} + 0.65x_{t-2} - \varepsilon_t & \text{if } \lvert x_{t-1}\rvert > 1. \end{cases}$

3.2. Additional Complexities

To add additional complexity to the analysis, we have incorporated settings with a jump process and a random walk [48] into each DGP. The jump process captures sudden, abrupt changes in the system's behavior, representing regime shifts that may occur in logistics due to unforeseeable events such as supply chain disruptions, equipment failures, or market shocks [79,80]. For example, sudden demand surges during a pandemic or temporary halts in operations due to extreme weather events are real-world analogs of such jumps. The random walk, by contrast, models persistent, stochastic variations that add noise to the data, reflecting phenomena such as cumulative forecasting errors, drifting demand trends, or inaccuracies in inventory measurements [48,81]. These complexities are particularly relevant to logistics scenarios where external factors introduce substantial uncertainty and variability. Our study considers four different scenarios: (1) the DGP without additional complexity, (2) the DGP superposed with the jump process, (3) the DGP superposed with random noise, and (4) the DGP superposed with both the jump process and random noise. The *jumps* are modeled using a compound Poisson process $\{p_t\}_t$ [79]. The original DGP $\{x_t\}_t$ is then superposed by p_t as follows:

$$x_t^* = x_t + p_t,$$

where x_t^* denotes the resulting DGP, and the compound Poisson process is given by

$$p_t = \sum_{i=1}^{N_t} Z_i,$$

where N_t follows a Poisson distribution with parameter λ and $Z_i \sim \mathcal{N}(0, \sigma_p^2)$. For the jump experiments, we set σ_p^2 to 1. A larger σ_p^2 results in larger jumps in magnitude, while the mean over positive and negative jumps remains zero. The parameter λ is set to $\frac{n}{10}$, where n denotes the length of the generated time series. This means that, on average, a jump is expected to occur after every λ period. Superposing the DGP with the compound Poisson process results in a mean shift by the actual jump size that occurred at each jump event. As mentioned before, the noise is modeled by a *random walk* $\{w_t\}_t$ with

$$w_t = w_{t-1} + e_t,$$

where $e_t \sim \mathcal{N}(0, \sigma_{rw}^2)$. In our study, we choose σ_{rw}^2 in such a way that we obtain a setting with medium noise, i.e., a signal-to-noise ratio (SNR) of four. The SNR [82] is a measure that characterizes the strength of the signal relative to the background noise. A higher SNR indicates a clearer and more discernible signal amidst the noise. By including the random walk, we achieve a resulting DGP that is globally nonstationary due to the random walk overlay.

3.3. Additional Queueing Models

Beyond these 48 simulation models, we include the M/M/1 and M/M/2 queueing models [83] in our study. Queueing models are commonly used in logistics, operations research, and industrial engineering to study the behavior of waiting lines or queues [84–87]. Both models have numerous real-world applications, such as in call centers [88], healthcare facilities [89], and transportation systems [90]. The M/M/1 model is a classic queueing model that assumes a single queue and one server. It is a stochastic model, where customer arrivals are assumed to follow a Poisson process, and service times are exponentially distributed. The M/M/1 model can be used to analyze the expected waiting time, the number of customers in the queue, and the expected server utilization. The M/M/2 model is a variation of the M/M/1 model that assumes two parallel servers. According to [87], we set the arrival rate to four and the service rate to two. We focus on the complete queueing model, including both the arrival process and service process, to capture the full system behavior.

3.4. Number of Different Settings

For each setting, we generate time series of length n from the respective DGPs with $n \in \{100, 500, 1000\}$. In total, this results in 150 (=12 (time series DGPs) $\times$ 4 (further complexity) $+$ 2 queueing models) $\times$ 3 (lengths)) different simulation settings for each forecasting method.

3.5. Data Preprocessing

To forecast time series using a machine learning algorithm, we use the sliding window approach [91]. In this method, a fixed-sized window is moved over the time series data, where the data within each window are used as input for model training at each step. One key advantage of the sliding window approach is that it allows the machine learning algorithm to capture the temporal dependencies and patterns in the data. The window size is an important parameter [92]; if it is too small, it may not capture enough information, whereas, if it is too large, it may introduce noise and reduce the model's accuracy. In this

study, we evaluate window sizes of 2, 4, 8, and 16, examining their impact on forecasting performance for different time series lengths (100, 500, and 1000). We focus on one-step ahead forecasting at each time step, using both the original time series and the differentiated time series as input. Differencing is essential as it enhances stationarity and prevents the model from forecasting beyond the observed range, which trees cannot handle effectively.

3.6. Choice of Parameters

In this study, we applied different strategies for parameter selection based on the nature of the models. For machine learning models, we used default hyperparameter settings as recommended in the literature [56,66,67]. This decision was made to focus on their baseline performance and ensure consistency across comparisons while also reducing computational runtime. Specifically, each ensemble learner was configured with 500 trees, the inner bootstrap sample is equal to $m_{try} = \lfloor \frac{p}{3} \rfloor$, where p denotes the number of features, and the number of sample points in the bagging step is equal to the sample size. Each terminal node should at least contain five observations. For XGBoost, we employed a learning rate of 0.3 and a maximum tree depth of six. In contrast, to estimate the parameters of the time series approaches, we use the algorithms implemented in the R-package `forecast`. This was necessary to tailor these models to the specific properties of the data, such as trend and seasonality, as their performance heavily depends on optimized parameters.

3.7. Evaluation Measure

Since the mean square error (MSE) and the mean absolute percentage error (MAPE) are widely used in the forecasting of time series in logistics [9], we use them as evaluation measures, which are calculated over 1000 repeated forecasting steps. The MSE measures the model's accuracy, expressed as the average squared difference between the observed and predicted values. Simultaneously, the MAPE, calculated as the average percentage difference between the observed and predicted values, offers insights into the model's relative performance.

4. Results

In this section, we describe the results of the simulation study. In particular, we present the MSE of the different forecasting algorithms under various simulation configurations. The analysis of the MAPE results can be found in Appendix A. We start with the performance of the methods for queueing models.

4.1. Predictive Power in Queueing Models

The influence of the different sliding window sizes and the differencing is shown in Figures 1 and 2. Generally, differencing improves the prediction power of both ML approaches in both settings. Especially for the Random Forest, the MSE decreases by one-fifth after differencing. The lengths of the time series only have a minor influence on the MSE. The Random Forest with differentiated data outperformed the other methods for all lengths. Comparing the effects of sliding window sizes, we find slight differences in performance. Random Forests have smaller MSE values with smaller sliding windows in both settings, while larger window sizes slightly improve performance in the other approaches.

The predictive power of the time series and naive approaches are given in Figure 3. Note that both the ARIMA and SARIMA models have identical MSE values. In both cases, the time series approach performs better than the naive approach. However, the difference in performance is smaller for M/M/2. Again, the influence of the time series length is marginal. While all time series approaches perform similarly in the M/M/1 setting, the TBATS method has slightly smaller values in the M/M/2 setting.

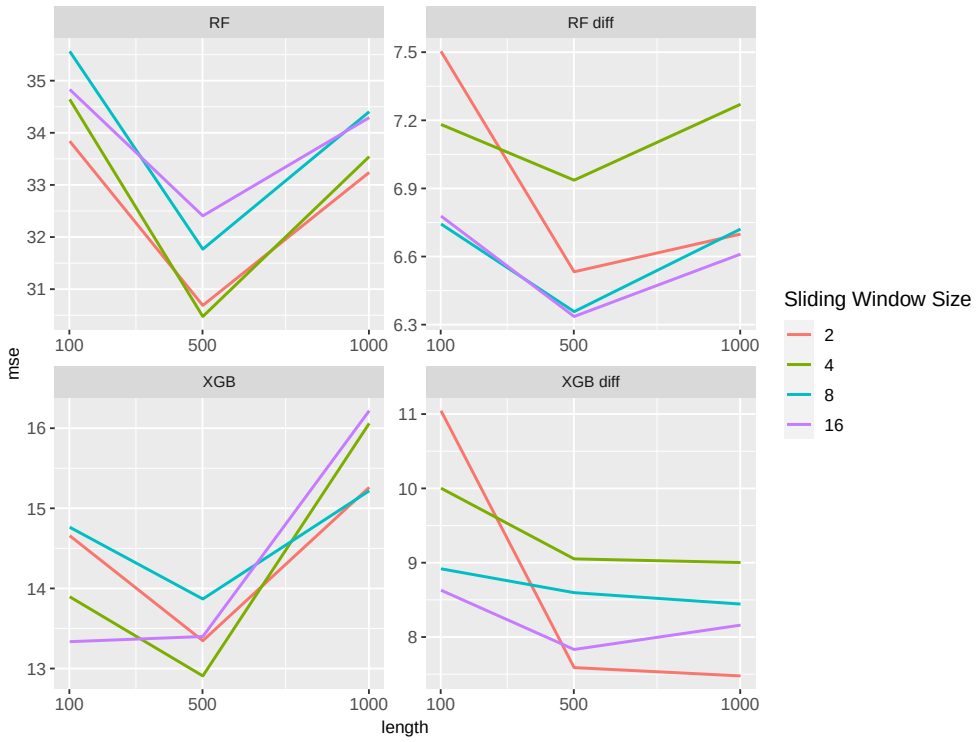

Figure 1. MSE of ML approaches separated by the sliding window size for the M/M/1 setting. XGB stands for XGBoost and RF for Random Forest; diff in the method name indicates that the data were differentiated.

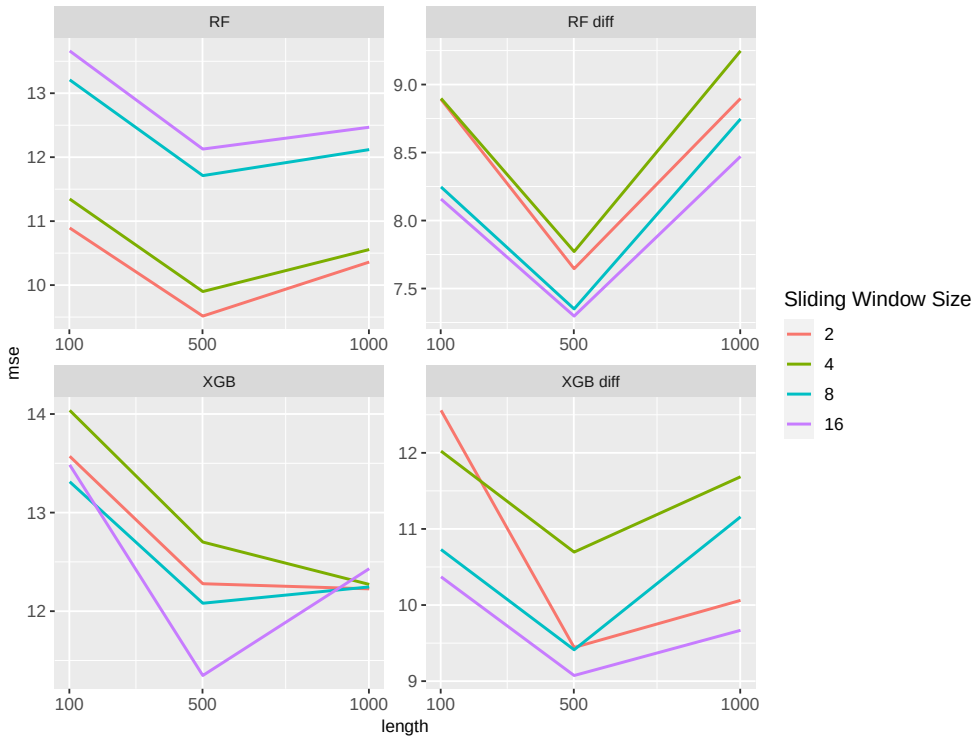

Figure 2. MSE of ML approaches separated by the sliding window size for the M/M/2 setting. XGB stands for XGBoost and RF for Random Forest; diff in the method name indicates that the data were differentiated.

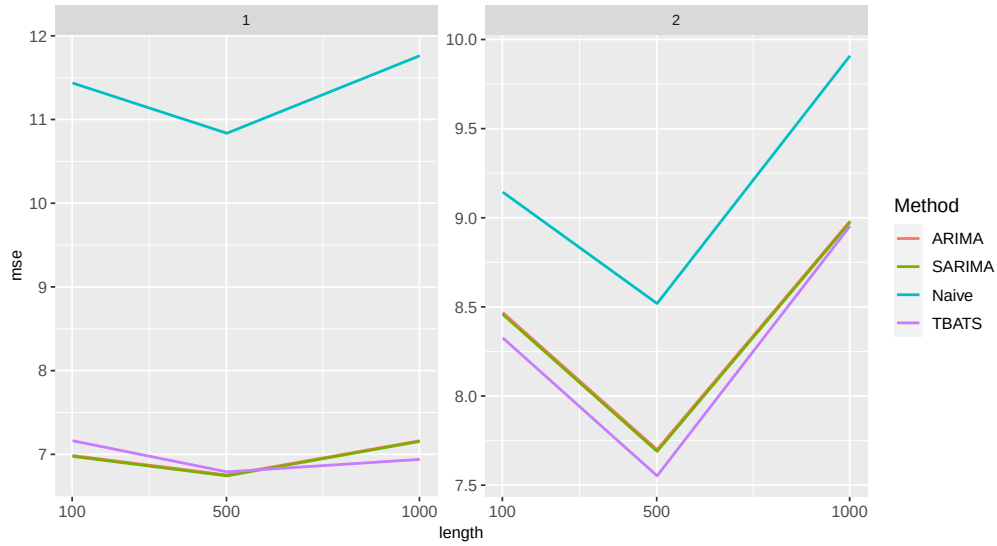

Figure 3. MSE of time series and naive approaches for the M/M/1 (**left**) and M/M/2 (**right**) setting. ARIMA and SARIMA models have identical MSE values, as no seasonality was present.

In both scenarios, the Random Forest approach with differenced data consistently showed the smallest MSE. However, the differences between this method and the time series approaches were not great.

4.2. Predictive Power in the Different Time Series Settings

In the following, we analyze the performance of the methods for the DGPs described in Table 1. When comparing the influence of sliding window size and differencing on the performance of Random Forest across all settings (Figure 4), we observed that non-differencing resulted in smaller MSE values except for the AR setting.

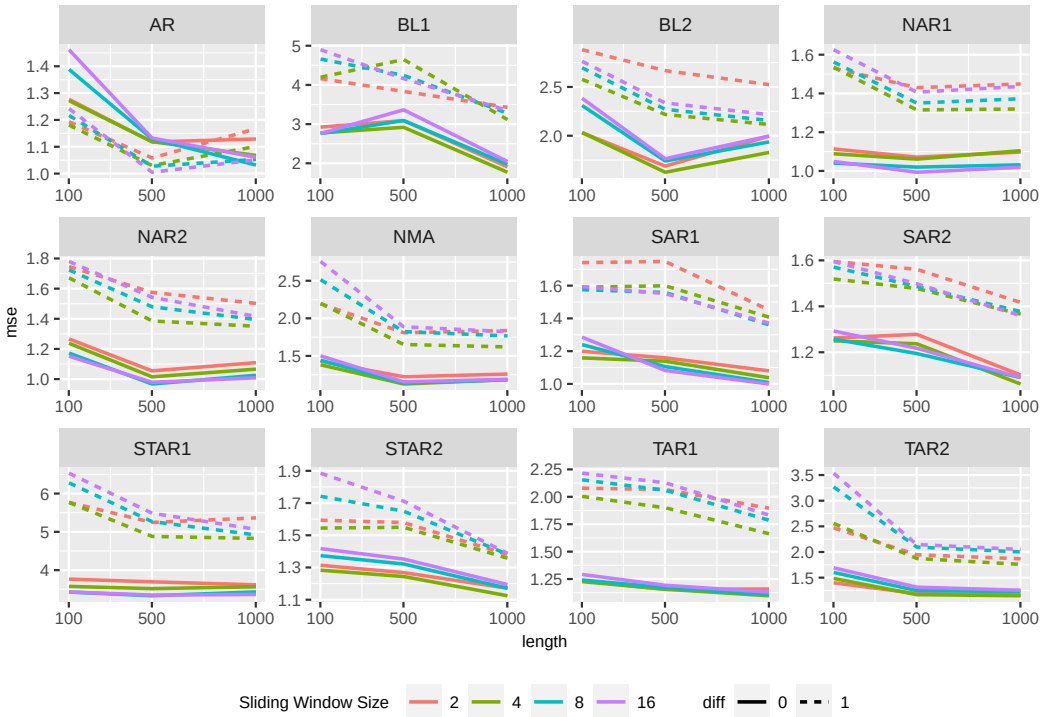

Figure 4. MSE of the Random Forest approaches separated by the sliding window size and differencing for the different data generating processes.

In the AR setting, differencing slightly outperformed non-differencing. However, it should be noted that, as the length of the time series increases, the differences between the two approaches become negligible. In all settings, the MSE values slightly decrease with an increase in time series length. The sliding window size has a small influence on the prediction power and shows similar behavior across different time series lengths.

Similar observations can be made for XGBoost, see Figure 5.

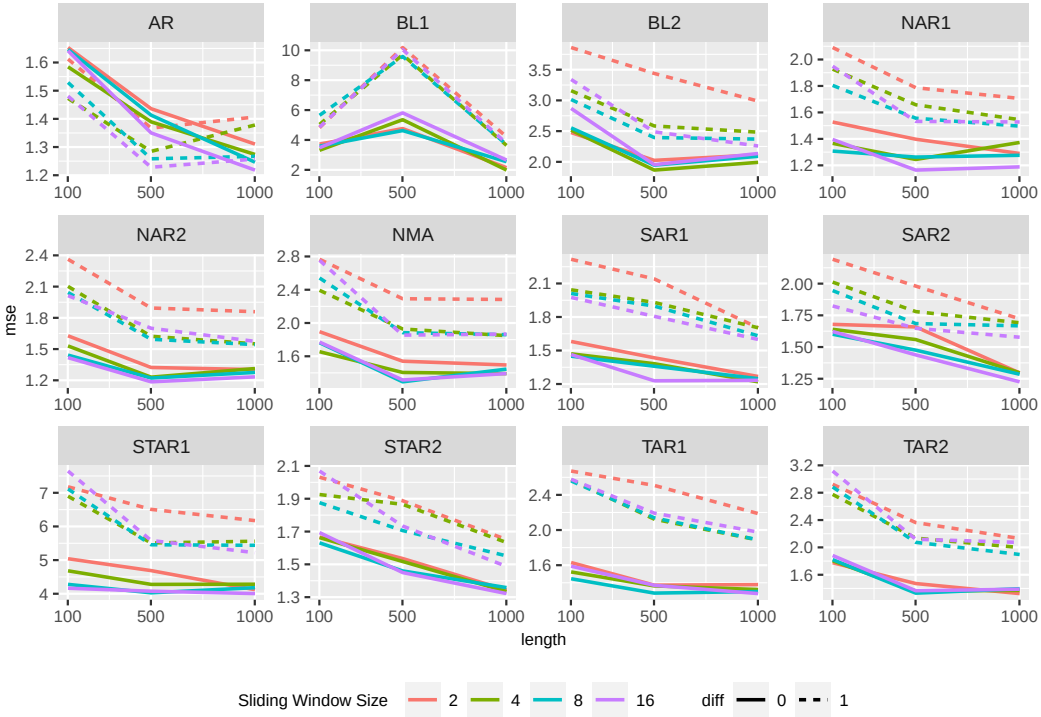

Figure 5. MSE of XGBoost approaches separated by the sliding window size and differencing for the different data generating processes.

The sliding window's size and the time series length have a small effect on the performance quality. For all DGPs, the MSE values decrease slightly with increasing time series length, except for BL1. Here, the MSE values first increase. The XGBoost approaches generally have slightly larger MSE values than the Random Forest approaches.

Figure 6 shows the MSE values for the time series approaches. The performance of the time series approaches is comparable to that of the Random Forest. All methods have very similar MSE values. The time series length has only a minor impact on the predictive power, except for the BL1 setting. As observed for the XGBoost approaches, MSE values in this setting first increase and then decrease with increasing time series length.

Additional results can be found in the Appendix A Figure A1 therein, for example, shows that the naive approach exhibits the largest MSE values compared to all methods. Thereby the performance of the naive approach is dependent on the DGP and the length of the time series. For BL2, longer time series lengths generally lead to better performance, but for NAR1 the performance may slightly decrease. For the AR, BL1, and NMA models, the MSE values typically decrease initially and then slightly increase as the time series length increases. Conversely, NAR2, SAR1, SAR2, STAR1, STAR2, TAR1, and TAR2 tend to show the opposite trend.

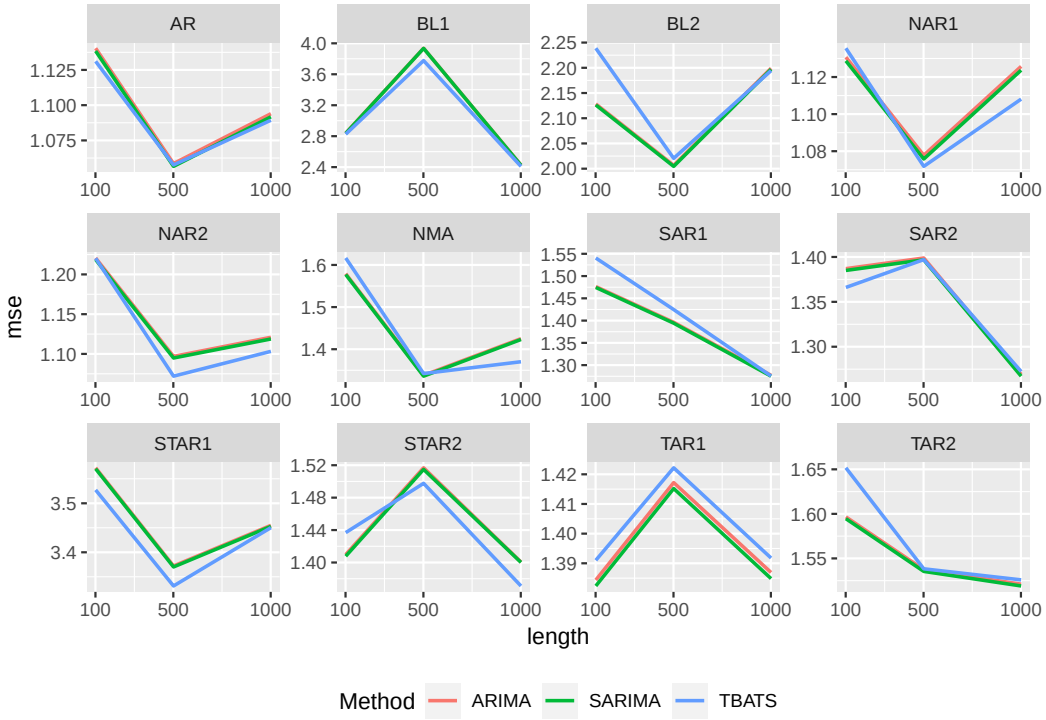

Figure 6. MSE of the time series approaches for the different data generating processes.

4.3. Influence of the Additional Complexities on the Predictive Power

Based on the findings of the previous sections, we focus on the simulation results obtained with a sliding window size of 8, as the choice of this size is due to the consistent performance observed with different sizes. Details of the results with other window sizes can be found in Appendix A, but a moderate size of 8 balances computational efficiency and information incorporation. Below, we first consider the influence of an additional jump process before discussing the white noise results.

The influence of the jumping process can be seen in Figure 7. All MSE values increase monotonically with increasing sample size, indicating that the jump process significantly impacts predictive performance. Note that, as the time series length increases, the Random Forest approach with differentiated data outperforms all other approaches. Using the differenced data significantly improves the MSE values for both ML approaches, particularly for increasing time series length. The predictive performance of the time series approaches is similar for all DGPs and slightly better than that of the naive approach.

Figure 8 summarizes the prediction results for all methods and all DGPs superposed by a random walk. Here, the time series length has only a minor influence on the prediction performance of the data overlaid with a random walk. For the AR and BL2 settings, the MSE values increase slightly when the time series length is increased from 100 to 500. For all other DGPs, the MSE values decrease slightly, except for the naive approach. The naive approach has the highest MSE values for all settings, followed by XGBoost, except for BL2. Here, both approaches have similar values. The performance of the other methods depends on the respective setting.

For the settings AR, BL2, SAR1, and SAR2, Random Forest with differenced data again shows the smallest MSE values, while the time series approaches show slightly larger values. Note that the XGBoosts with differentiated data perform better in these settings than the Random Forests with non-differentiated data. In the BL1, NAR1, NAR2, NMA, and STAR2 settings, only minor differences in the performance of the Random Forests and time series approaches can be observed. When comparing the two XGBoost approaches in these

settings, the differencing reduces the MSE. The ML approaches show larger MSE values in the STAR1, TAR1, and TAR2 settings than the time series approaches, with Random Forests performing better than the XGBoost method.

The influence of both complexities, the random walk and the Poisson process, on the prediction performance is shown in Figure A6 in Appendix A. Similarly to the case where a composite Poisson process is superposed on the data, we observe an increase in MSE values with increasing time series length for all settings. In particular, for time series lengths of 500, we obtain MSE values of more than 2000.

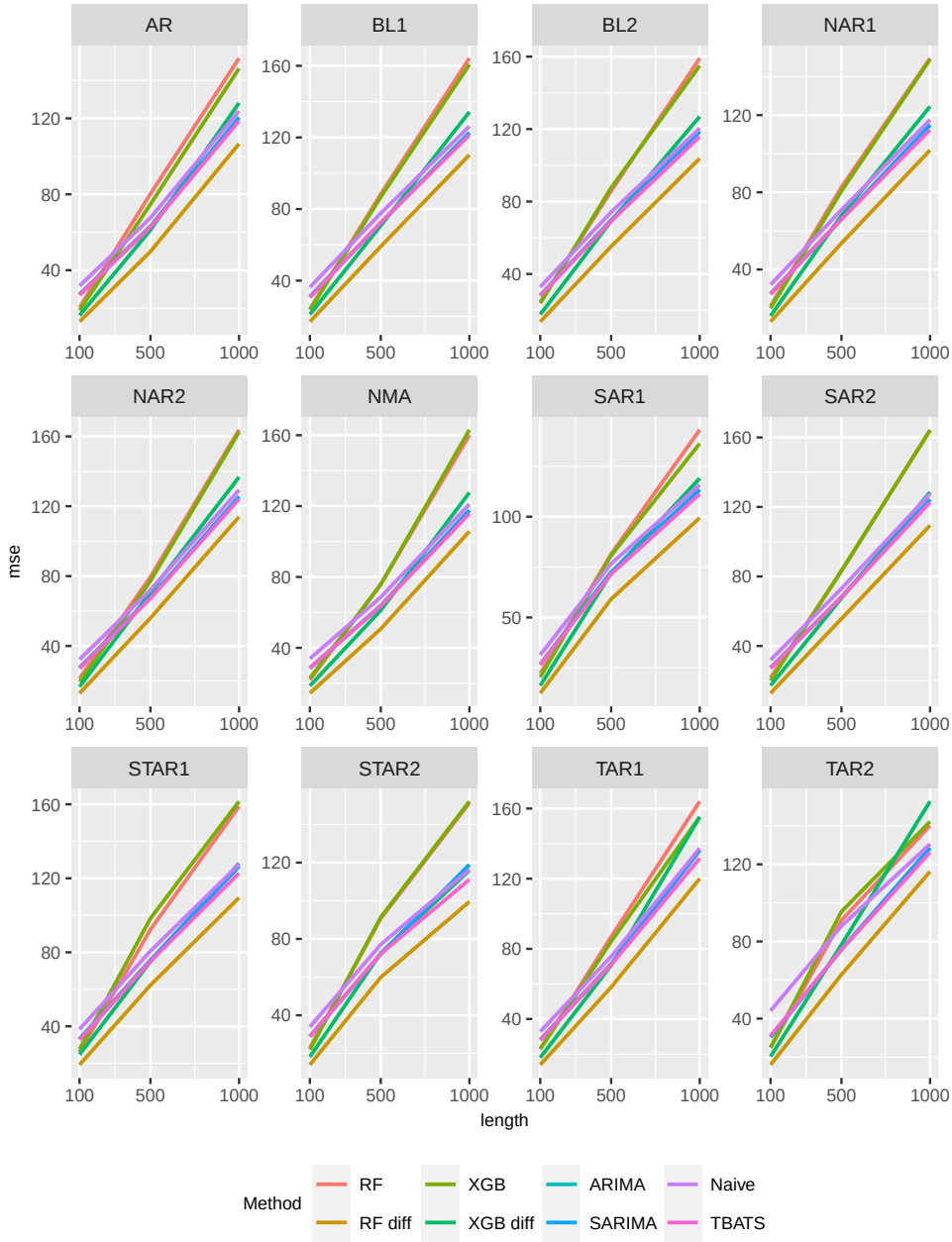

Figure 7. MSE values of all methods and data generating processes superposed by a compound Poisson process.

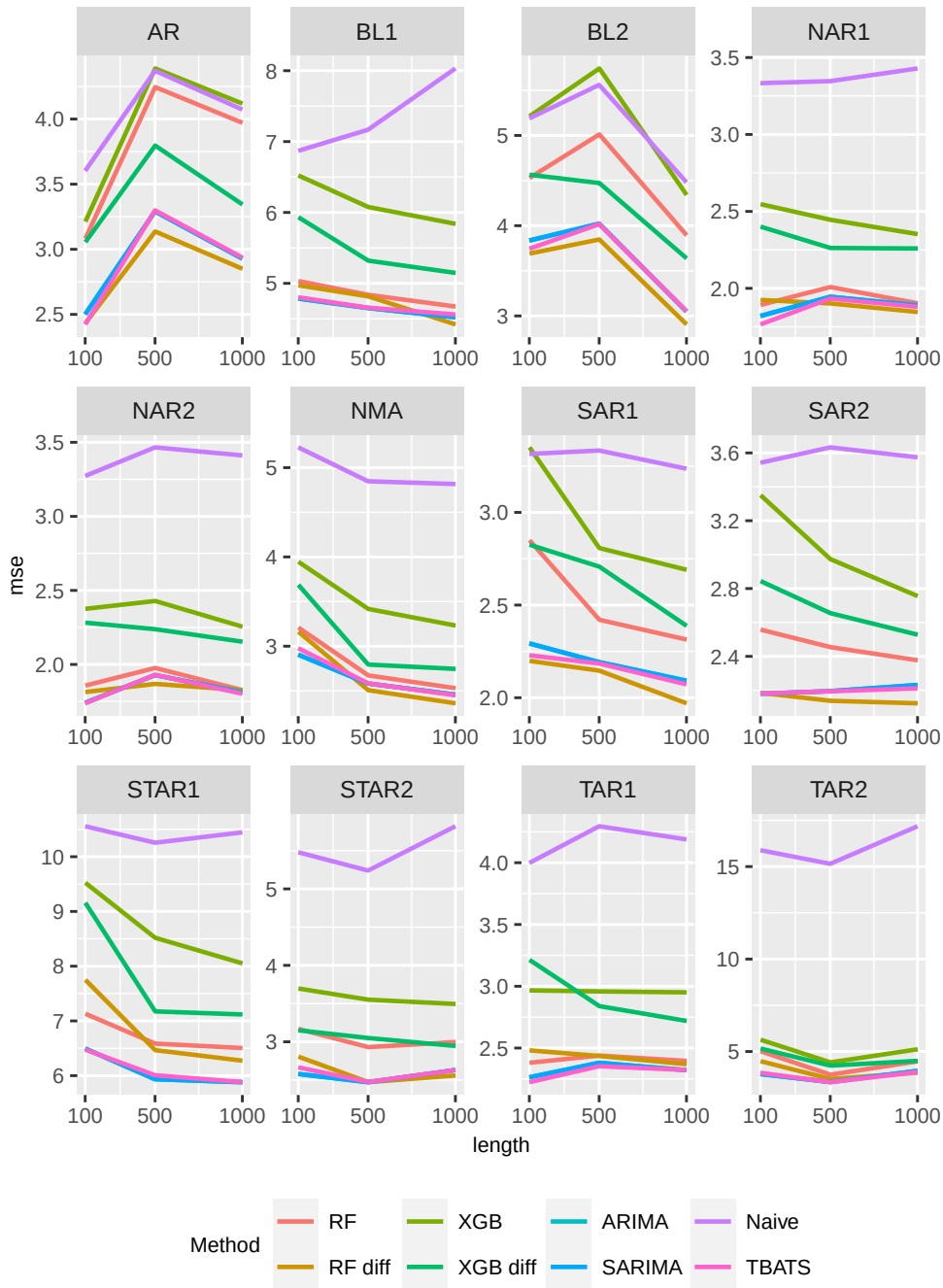

Figure 8. MSE values of all methods and data generating processes superposed by a random walk.

4.4. Summarizing All Results

To evaluate the prediction performance across the spectrum of simulation settings, we calculate the median rank for each prediction method in Table 2. The ranking is based on the MSE values, with rank 1 indicating the method with the lowest MSE. Each entry in the table represents the median rank of a particular prediction method in all settings of a particular DGP model described in Section 2. Furthermore, the results for the ranking take into account the performance of machine learning algorithms with a sliding window size of 8.

The results in Table 2 provide useful insights into the relative predictive performance of the different methods in different simulation scenarios. In particular, Random Forest with differentiated inputs proves to be the best performing method, achieving the lowest median value across different complexities, including scenarios with jumps, random walks,

or a combination of both. While XGBoost is competitive, it tends to have a slightly higher median value under these conditions. Traditional time series methods such as ARIMA, SARIMA, and TBATS consistently show a robust and similar performance.

Table 2. Median performance rank of forecasting methods across different simulation settings and different time series lengths. Rankings are based on MSE values, with rank 1 indicating the method with the lowest MSE.

DGP		RF	RF Diff	XGBoost	XGBoost Diff	ARIMA	SARIMA	TBATS	Naive
Queueing Models		7	1	7	5	2.5	3.5	3	6
DGPS	no add. Compl.	1	6	5	7	3	3	3	8
from	Jumps	7	1	7	5	3	3	3	6
Table 1	Random Walks	5	1	7	6	3	3	3	8
with	Both	7	1	7	6	3	3	3	5

5. Real-World Data Example

As explained at the onset, there is a lack of freely available and good documented data sets in logistics research. We therefore use a rather simple real-world data example for illustration. The data set contains daily demand orders from a Brazilian logistics company [93] and was sourced from the UCI Machine Learning Repository [94]. Covering a span of 60 consecutive days, the data set consists of three time series that capture orders for products A, B, and C. Figure 9 shows the corresponding time series in which specific shocks in the data can be identified.

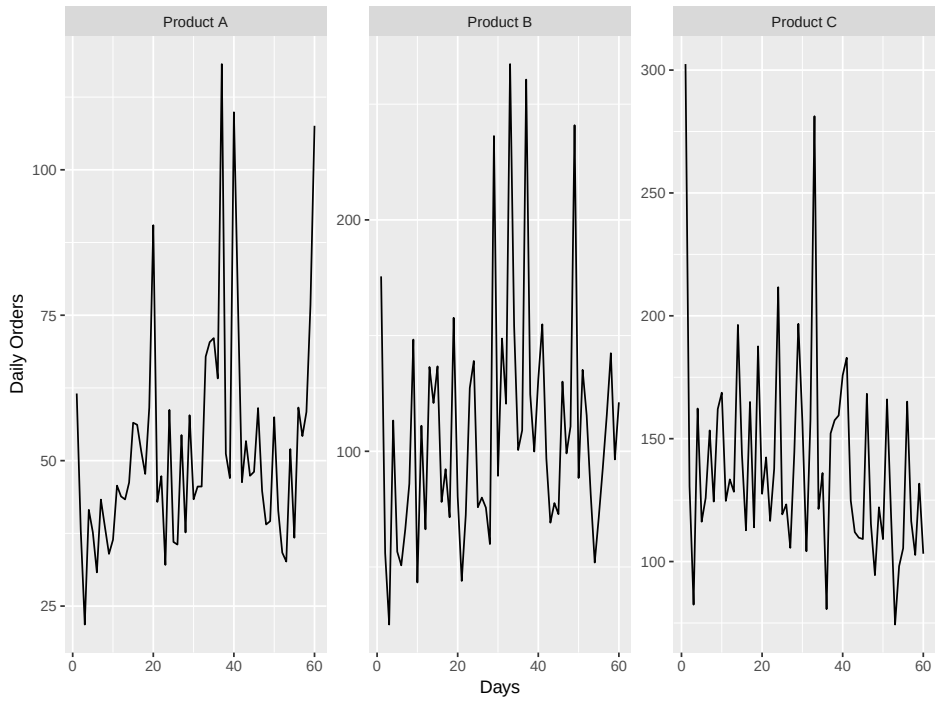

Figure 9. Daily orders of a Brazilian logistics company separated by the different products.

This observation puts us in a similar setting to the simulation study where the DGP was overlaid with a Poisson process. Given this context, it is of interest to evaluate whether the robust performance of (differentiated) machine learning algorithms observed in the simulation study is also apparent in this data set.

The machine learning algorithms adhere to the hyperparameters outlined in Section 3, with a sliding window size of eight, as informed by insights from our simulation study. We

use the first 50 observations to train all methods and the last ten observations to test the performance via time series crossvalidation ([43], Chapter 5.10). The MSE and MAPE are again used as evaluation measures. The summarized results are presented in Table 3. Note that the results of SARIMA and ARIMA are identical due to the absence of seasonality and are therefore combined into one method.

Table 3. Mean MAPE and MSE of the methods considered in Section 2 using daily demand order data set.

	MAPE			MSE		
Method	**Prod. A**	**Prod. B**	**Prod. C**	**Prod. A**	**Prod. B**	**Prod. C**
Random Forest	24.30	35.05	30.79	22.39	262.41	695.70
Random Forest Diff	6.67	21.80	15.84	4.91	197.23	1.97
XGBoost	25.06	41.62	19.51	22.34	376.62	147.20
XGBoost Diff	10.70	37.98	27.15	13.10	841.56	41.00
(S)ARIMA	28.57	49.30	33.56	29.48	1142.14	655.88
TBATS	28.37	36.17	33.56	43.14	446.18	663.78
Naive	33.18	30.71	30.59	25.10	194.21	82.03

The results show that the performance of the forecasting methods is different in the various product categories. In general, the machine learning algorithms deliver consistently better results than the traditional time series methods. This is in line with our simulation study, where ML methods showed better performances when additional complexities were present. Random Forest with differencing performed best for all three time series and evaluation measures, again confirming the results obtained in the simulation study for such settings. It should be noted that the introduction of differencing is beneficial for Random Forest in all predictions. For XGBoost, however, performance on product A improves significantly when differenced data is used, but in the other two time series differencing leads to worse forecasting performance.

6. Summary, Discussion, and Outlook

6.1. Summary with Highlights

The main objective of this simulation study was to perform a one-step comparative analysis of prediction accuracy and evaluate the performance of tree-based machine learning and time series approaches that are typically used in data-driven logistics. Through a comprehensive investigation of different data generating processes, queueing models, and additional complexities, we aimed to determine each method's inherent strengths and limitations. Our analysis included conventional time series methods, including (seasonal) ARIMA models and TBATS, as well as machine learning methods such as Random Forest and XGBoost. In addition, we investigated the impact of data differencing on the performance of the two latter algorithms. The key findings from our study are as follows:

- The out-of-the-box Random Forest emerged as the ML benchmark method.
- Training on differentiated time series can significantly improve the ML resilience.
- ML models are more robust with respect to additional (nonlinear) complexity, settings in which they outperformed statistical time series approaches.
- In all other settings, the time series approaches were at least competitive or even performed better.

6.2. Detailed Discussion and Outlook

In our study, the Random Forest approach performed consistently better in all simulation settings than the XGBoost approaches. It is worth noting that no hyperparameter

tuning was made in our study. Random Forests are known to be robust to hyperparameter settings and often perform well with default values [95,96]. This robustness can be a crucial factor contributing to their superior performance compared to XGBoost. Applying techniques such as Bayesian Optimization or more simple grid or random search for hyperparameter tuning could change this observation and should be investigated in future studies. Regarding the effect of data differencing on the performance of the two machine learning methods, we observed similar patterns. Differencing improved performance, especially in queueing scenarios and situations where additional complexity was introduced into the data generation process. Without additional complexity, differencing showed minimal impact, with the performance of both methods deteriorating slightly when the differentiated data were used, except for very linear data generation processes. Here, only a slight improvement was observed. This suggests that differencing plays a crucial role in improving the resilience of machine learning methods, especially Random Forests when the data is overlaid with additional noise like a random walk. When comparing the performance of the different time series approaches, we found subtle differences between them. ARIMA and SARIMA showed relatively similar performance in all simulation settings under consideration. Their prediction accuracy was quite consistent without big differences in most situations. Comparing their performance with that of TBATS, the differences are also small and not substantial, suggesting that ARIMA, SARIMA, and TBATS had comparable predictive power in our simulation settings. The additional complexity induced, such as a jump process or random noise, significantly impacts the predictive power. Introducing a jump process leads to increased MSE values for all methods and settings, indicating a significant impact on prediction accuracy. In this scenario, all methods show consistent behavior with strong increasing MSE values for increasing time series lengths. When a noise process is introduced, a more nuanced pattern emerges. For the machine learning approaches, differentiating the data proves beneficial and improves the overall performance. The Random Forest approach with differenced data as input outperforms the other approaches in most scenarios, closely followed by all three time series approaches. A comparison between Random Forests and the time series approaches shows different performance patterns in the different simulation environments. In queueing situations, where the underlying processes are often characterized by complicated dynamics, the Random Forest approach shows superior performance. Furthermore, a notable trend emerges in simulation settings where a Poisson process complements the data generating processes. In these cases, ML methods show improved performance, indicating robustness to the inherent complexity introduced by the Poisson process. The adaptability of ML models to capture and learn from nonlinear patterns may contribute to their effectiveness in scenarios with Poisson process or random walk overlays. However, it is essential to recognize that this beneficial performance of ML methods is not universal. In all other simulation settings, the Random Forest approaches perform comparably or slightly worse than all three time series approaches. In addition to the simulation study, our illustrative data analyses were conducted with a focus on one-step demand forecasting for different products of a logistics company. The results indicate that machine learning algorithms can improve the forecasting performance in this context. In particular, the machine learning methods perform better than or as well as the time series methods for most products.

In the context of data-driven logistics, our results underscore the importance of tailoring time series forecasting methods to the specific characteristics of data sets encountered in different logistics areas. The Random Forest approach, especially when using differentiated data as input, is recommended as an initial benchmark prediction tool, particularly for data sets with a lot of noise or complex patterns. The robustness of Random Forests, combined with their ability to achieve good results without extensive tuning of hyperparameters,

makes them a pragmatic choice for various prediction scenarios. Conversely, in situations where interpretability is paramount (e.g., to gain the understanding or trust of users in warehouses or decision makers in SCM) and the data exhibit clear patterns, traditional time series approaches remain a valuable and interpretable option. These approaches often come with faster runtimes and greater resource efficiency, which is also essential in the development of data-driven logistics, e.g., in the case of resource constraints [97,98]. As only one-step forecasts were considered, future simulation studies should investigate whether the same observations can be found for more step forecasting. Also, additional or hybrid methods must be investigated [99–101]. Another line of future research needs to compare the methods with respect to uncertainty quantification, i.e., point-wise or simultaneous prediction intervals and regions.

Author Contributions: Conceptualization, L.S. and M.P.; methodology, L.S. and M.P.; software, L.S.; validation, L.S. and M.P.; formal analysis, L.S. and M.P.; investigation, L.S.; writing—original draft preparation, L.S.; writing—review and editing, L.S., M.R., A.K. and M.P.; visualization, L.S.; supervision, M.P.; project administration, M.P. All authors have read and agreed to the published version of the manuscript.

Funding: The work of Lena Schmid and Markus Pauly was supported by the resKIL project, funded by the Federal Ministry of Food and Agriculture under grant number 28-D-K1.02F-20.

Institutional Review Board Statement: Not applicable.

Data Availability Statement: The real-world data set was obtained from the UCI Machine Learning Repository [94].

Acknowledgments: The authors gratefully acknowledge the computing time provided on the Linux HPC cluster at Technical University Dortmund (LiDO3), partially funded in the course of the Large-Scale Equipment Initiative by the German Research Foundation (DFG) as project 271512359.

Conflicts of Interest: The authors declare no conflicts of interest.

Appendix A

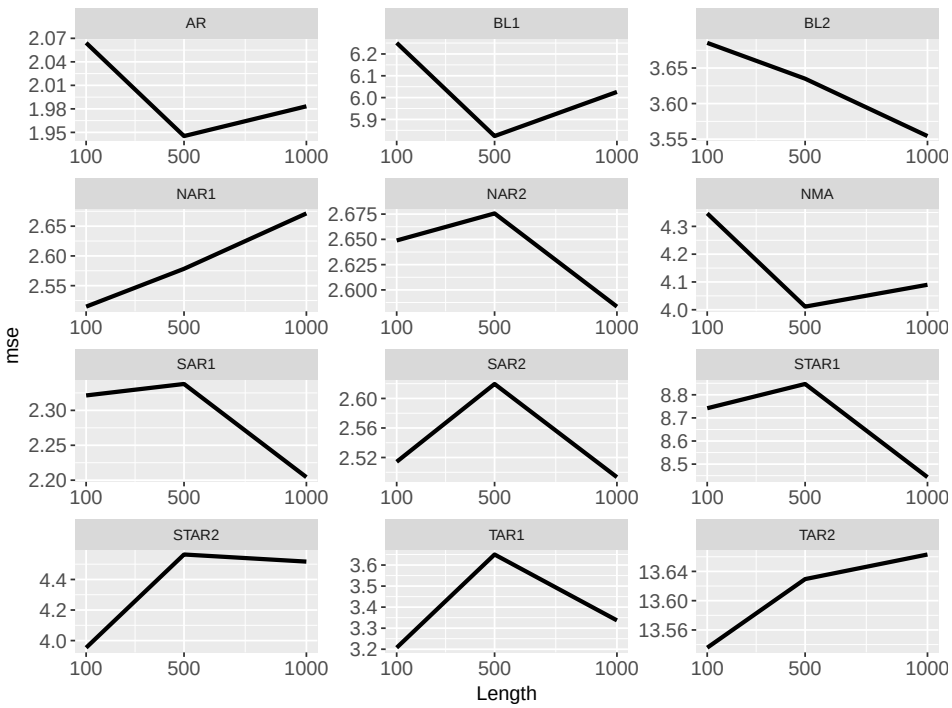

Figure A1. Averaged MSE of the naive approach for the different data generating processes.

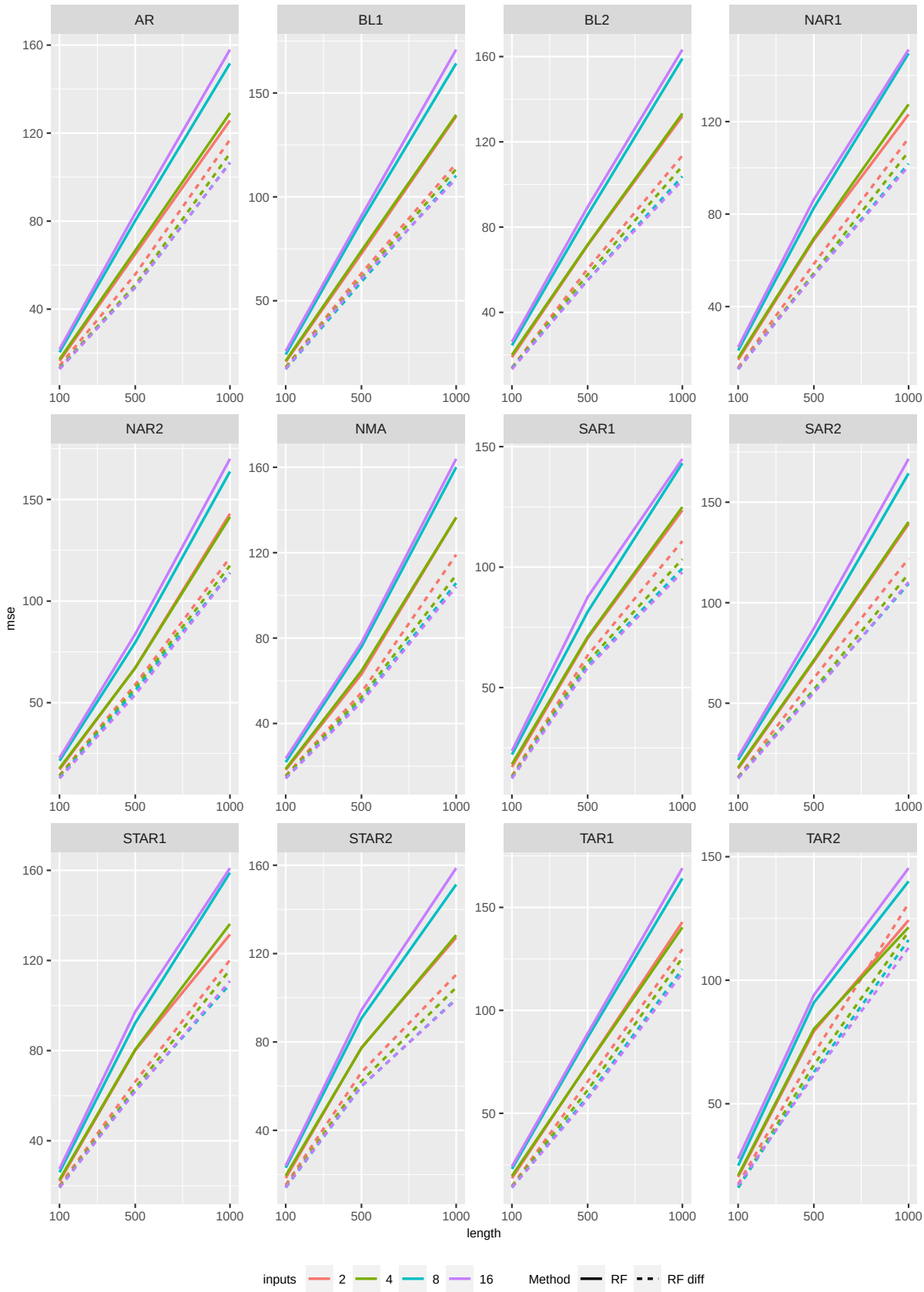

Figure A2. Averaged MSE values of all Random Forest approaches, sliding window sizes, and data generating processes superposed by a compound Poisson process.

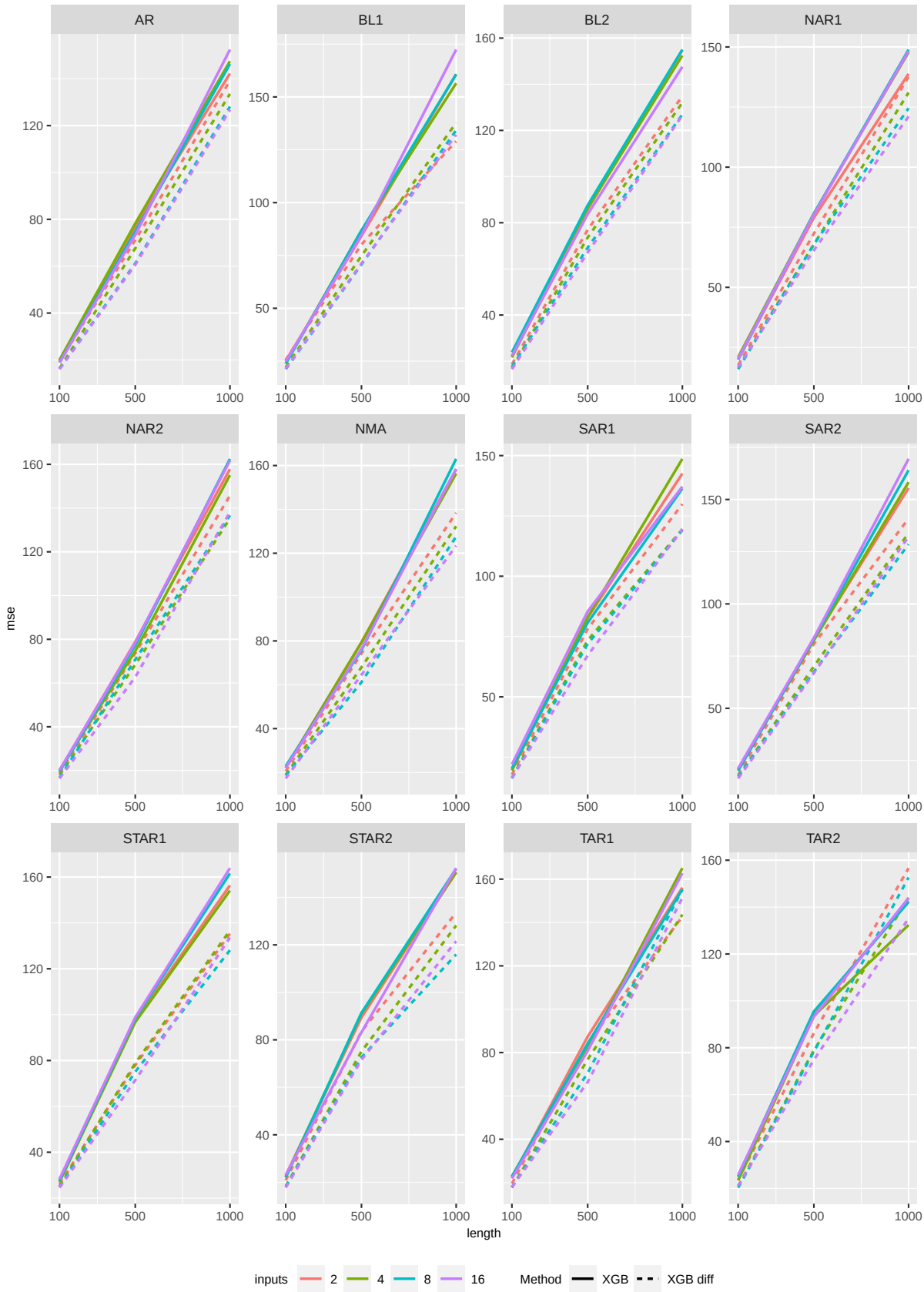

Figure A3. Averaged MSE values of all XGBoost approaches, sliding window sizes, and data generating processes superposed by a compound Poisson process.

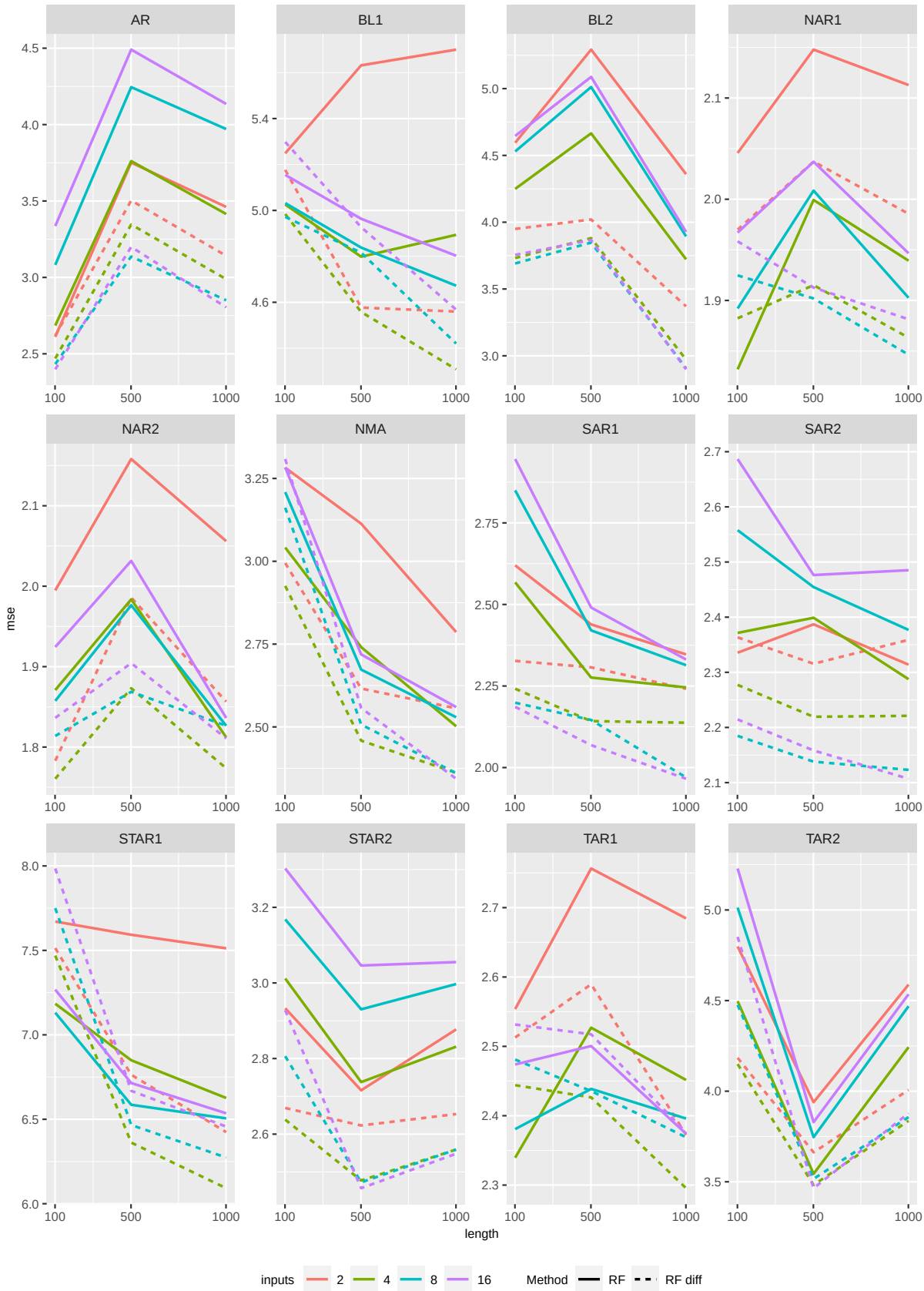

Figure A4. MSE values of all Random Forest approaches, sliding window sizes, and data generating processes superposed by a random walk.

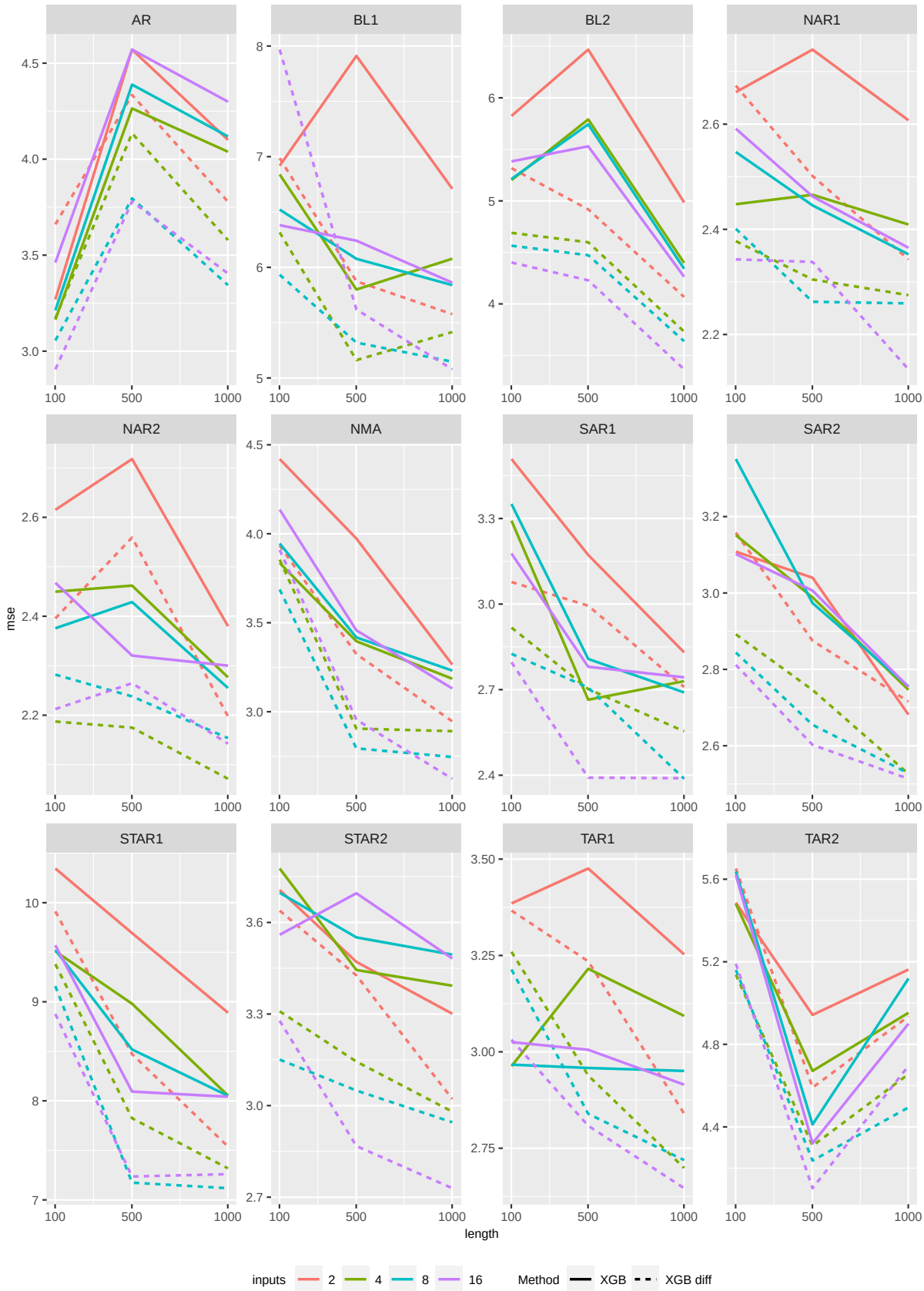

Figure A5. MSE values of all XGBoost approaches, sliding window sizes, and data generating processes superposed by a random walk.

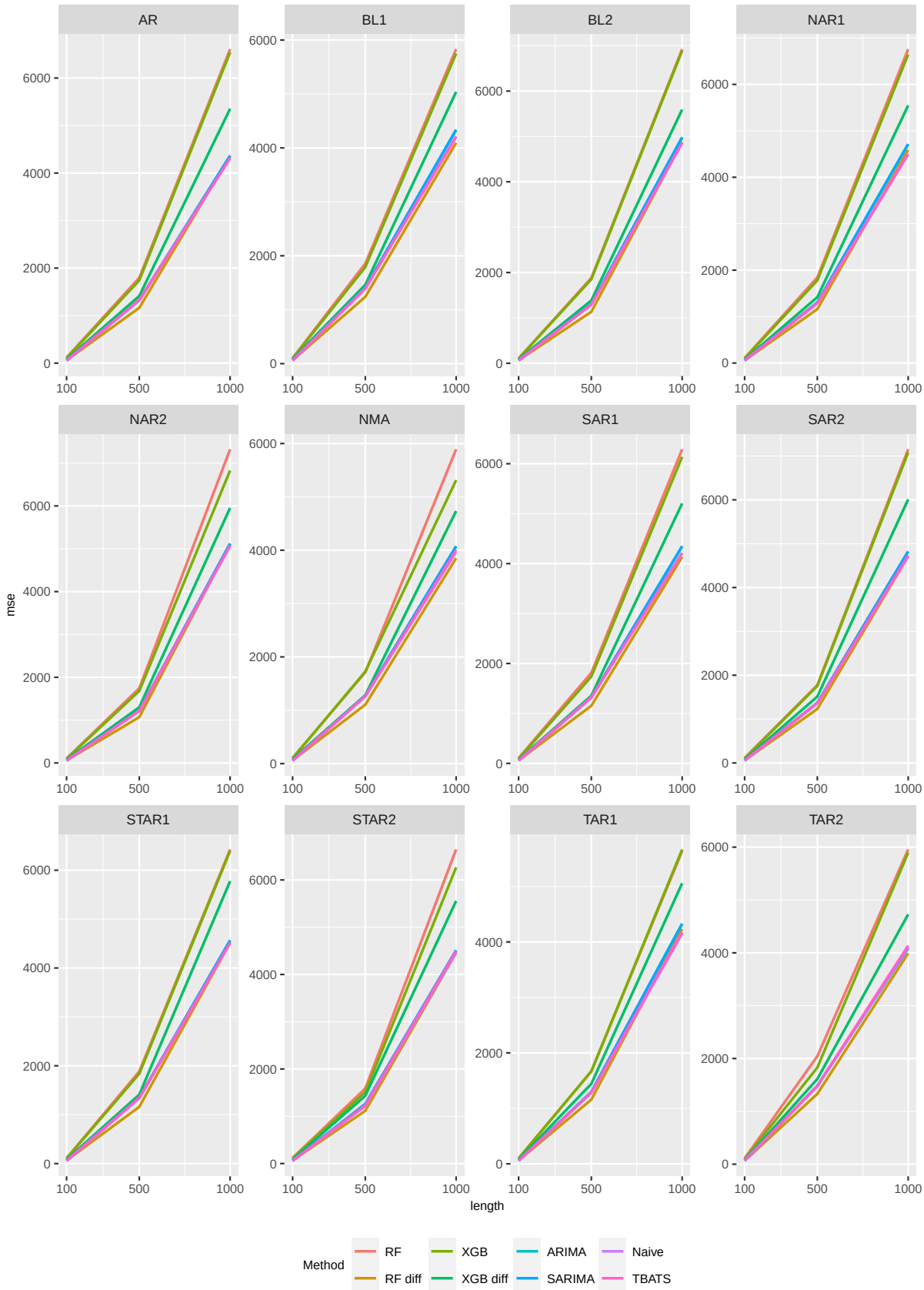

Figure A6. MSE of all methods and settings, where the data generating processes were superposed by a random walk and compound Poisson process.

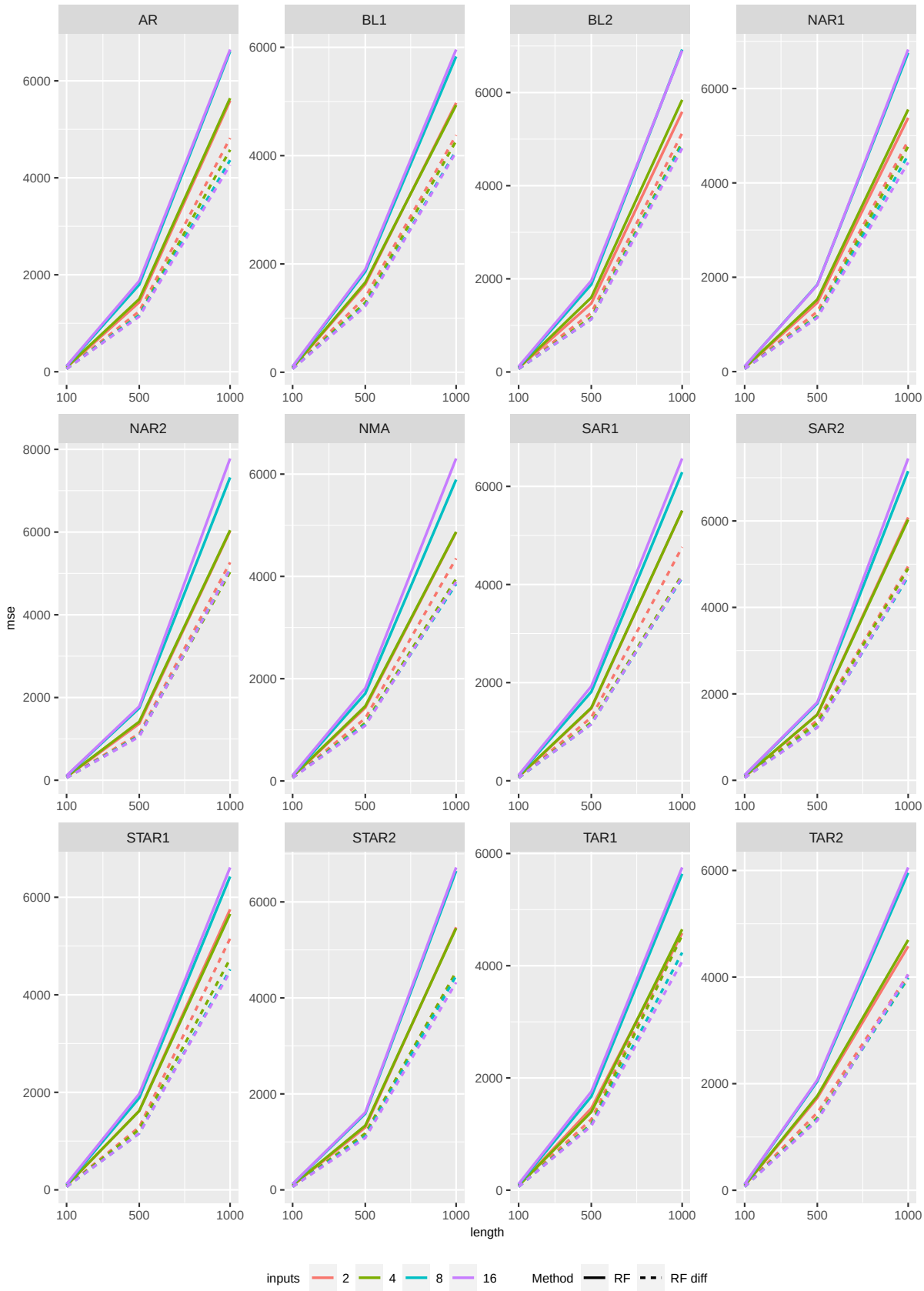

Figure A7. MSE of all Random Forest approaches, sliding window sizes, and settings, where the data generating processes were superposed by a random walk and compound Poisson process.

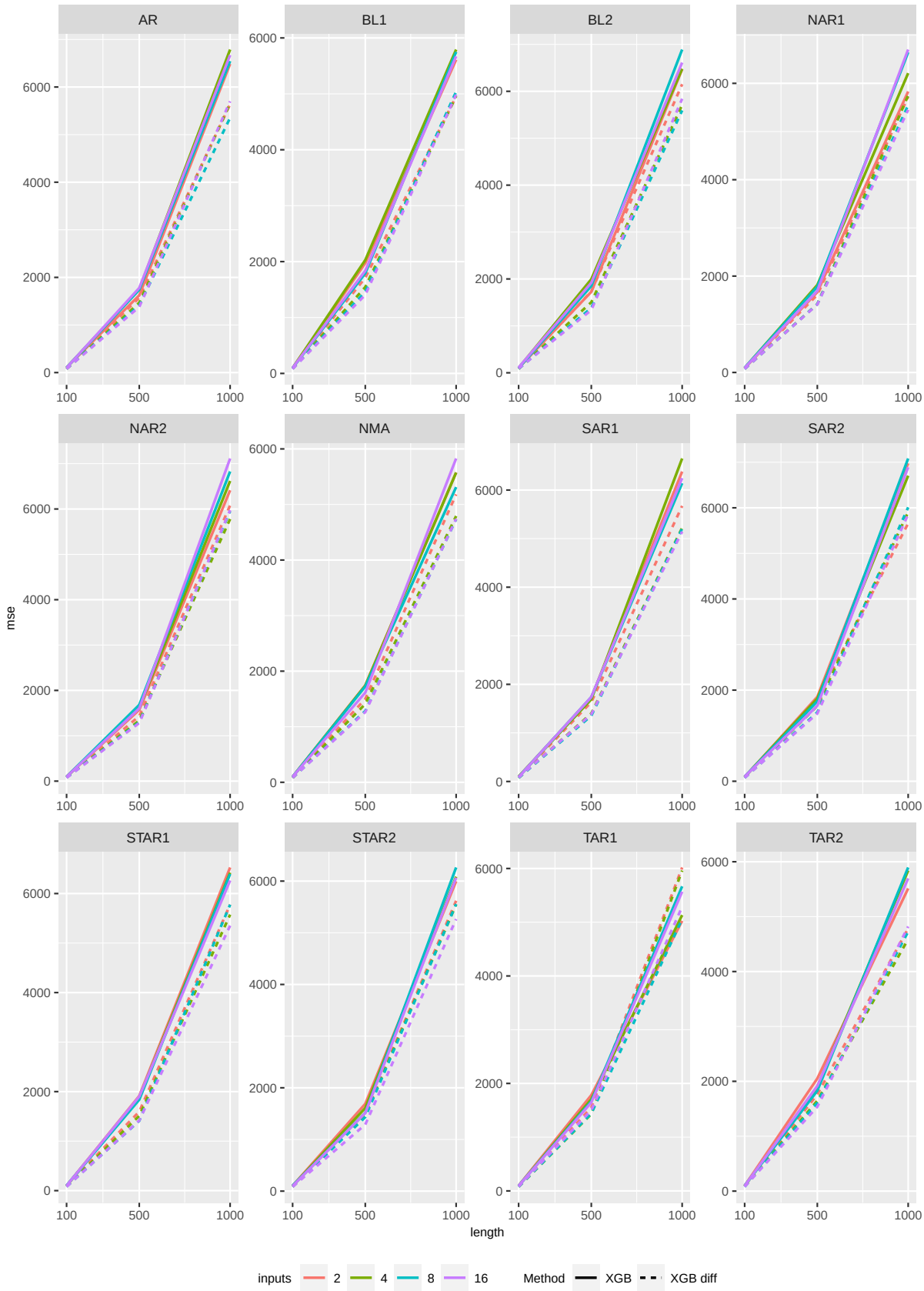

Figure A8. MSE of all XGBoost approaches, sliding window sizes, and settings, where the data generating processes were superposed by a random walk and compound Poisson process.

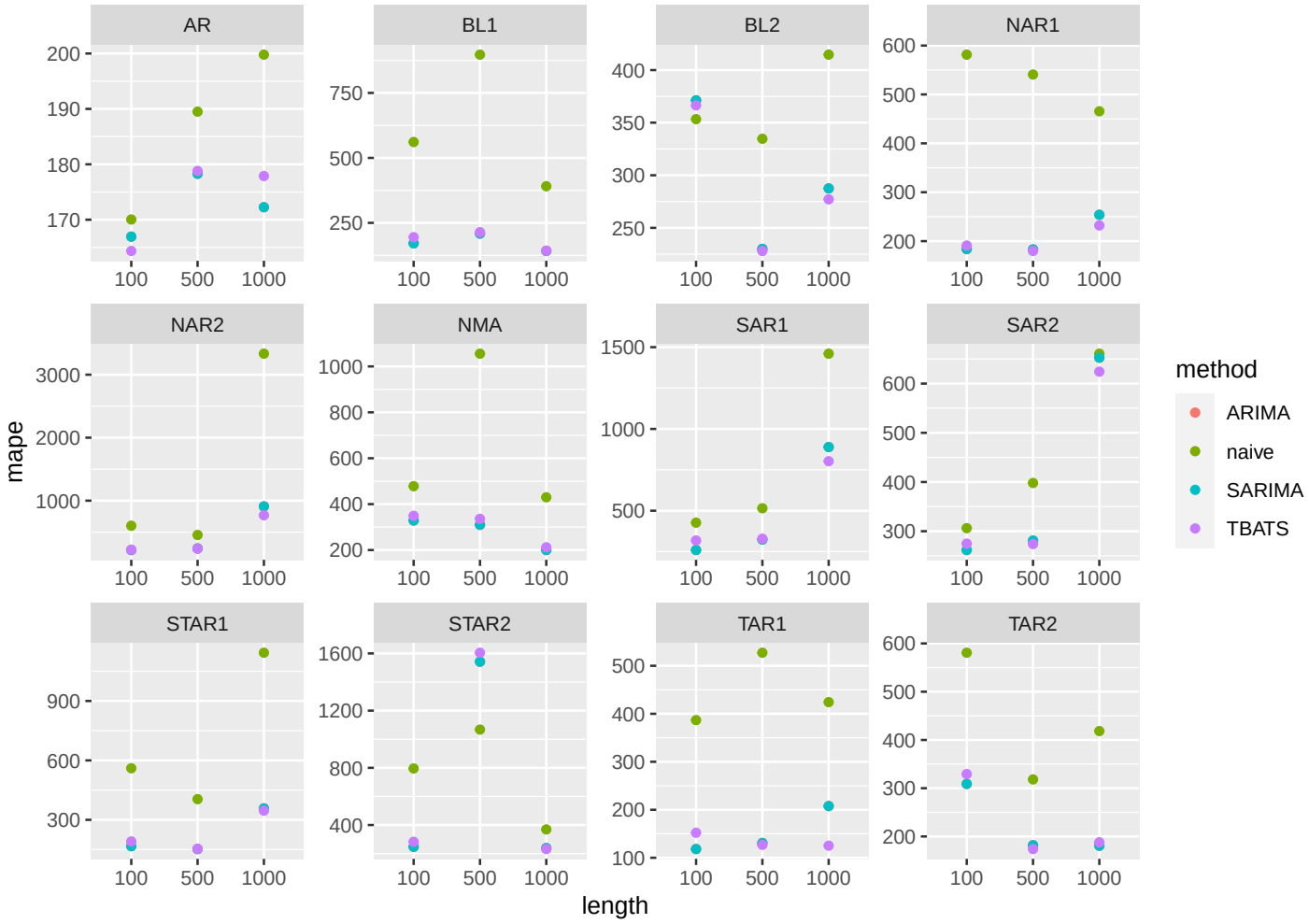

Figure A9. MAPE of the time series approaches for the different data generating processes described in Table 1.

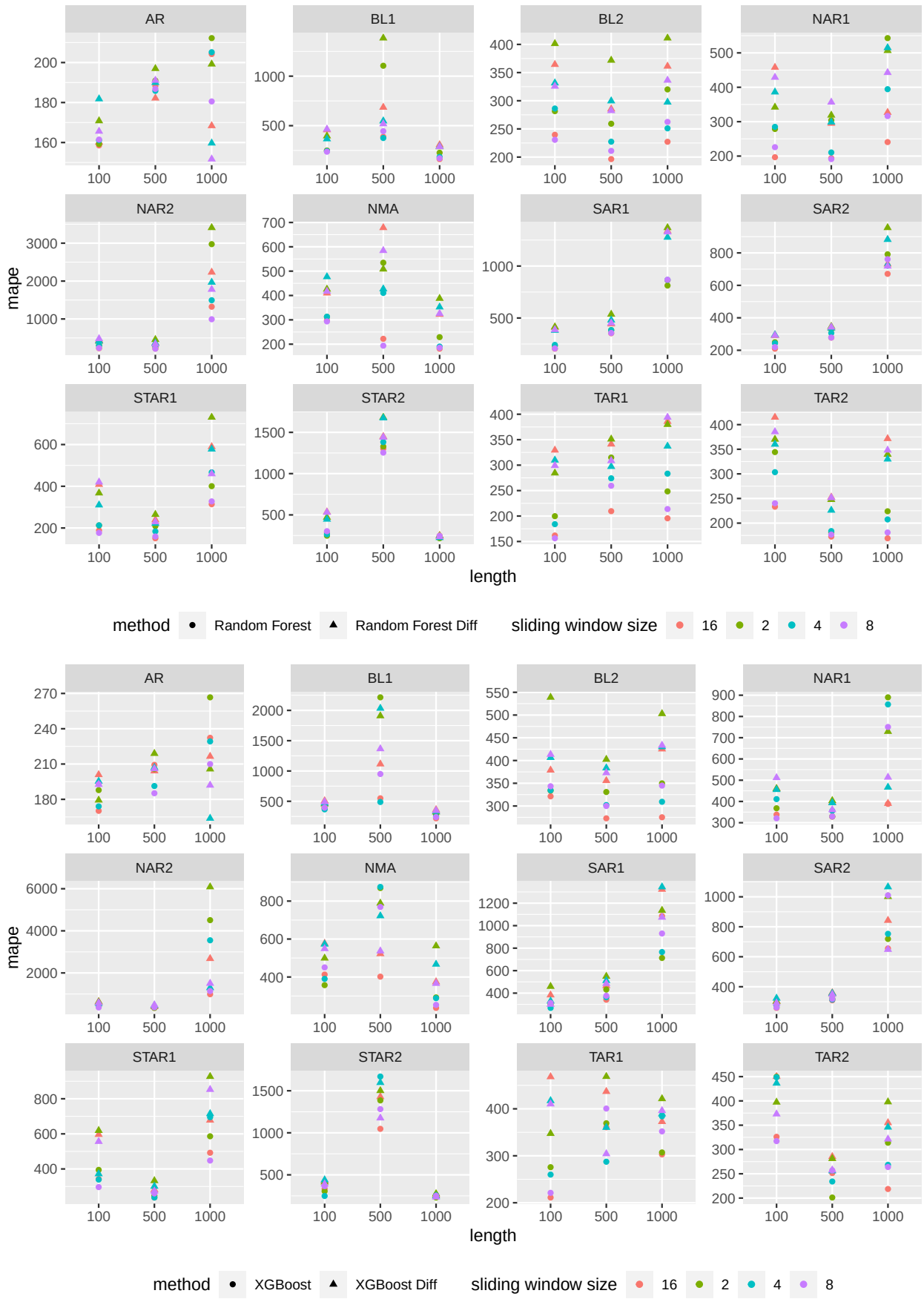

Figure A10. MAPE of the Random Forest (**above**) and XGBoost (**below**) approaches for the different data generating processes described in Table 1.

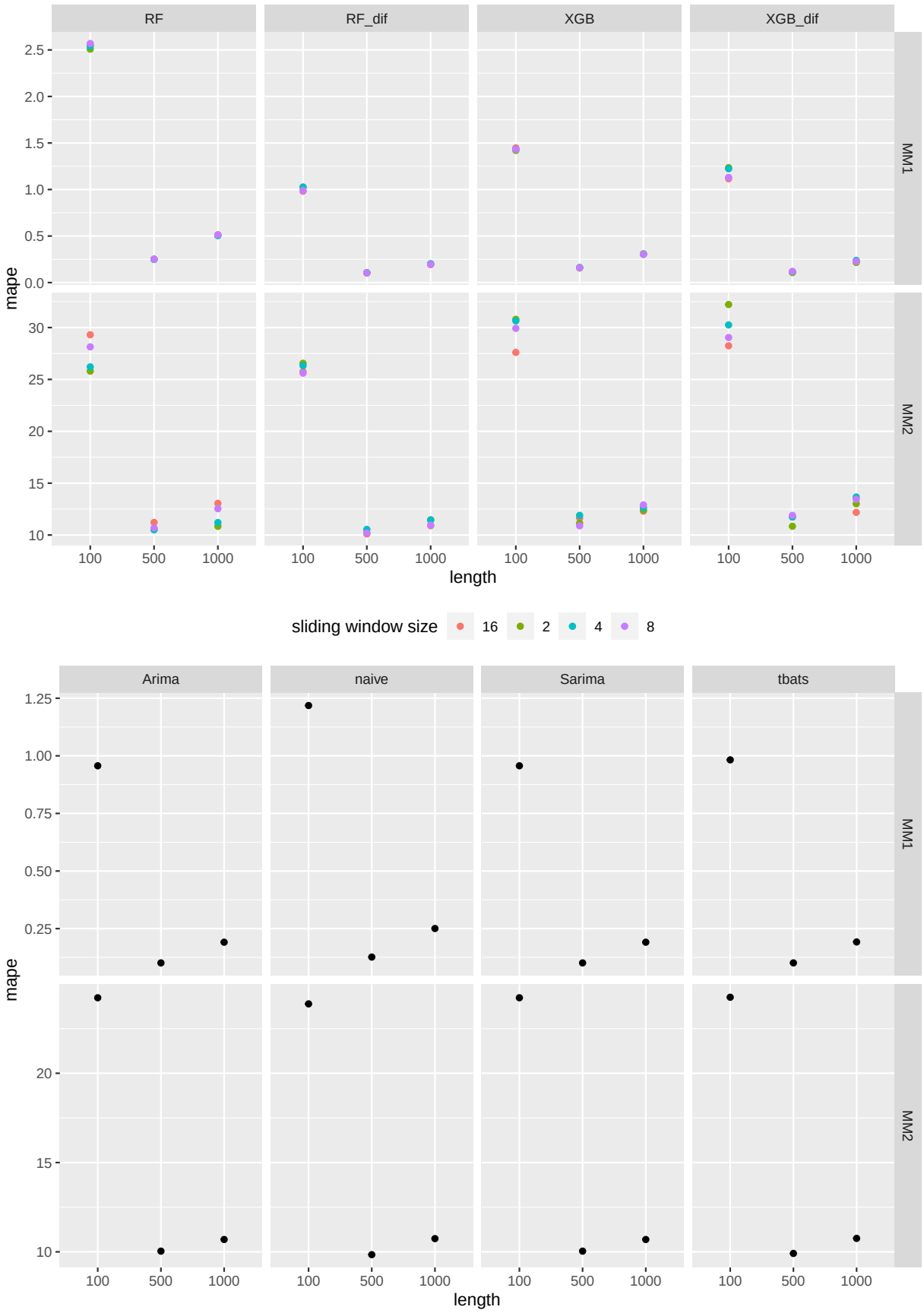

Figure A11. MAPE of the machine learning algorithms (**above**) and time series approaches (**below**) for the M/M/1 and M/M/2 data generating processes.

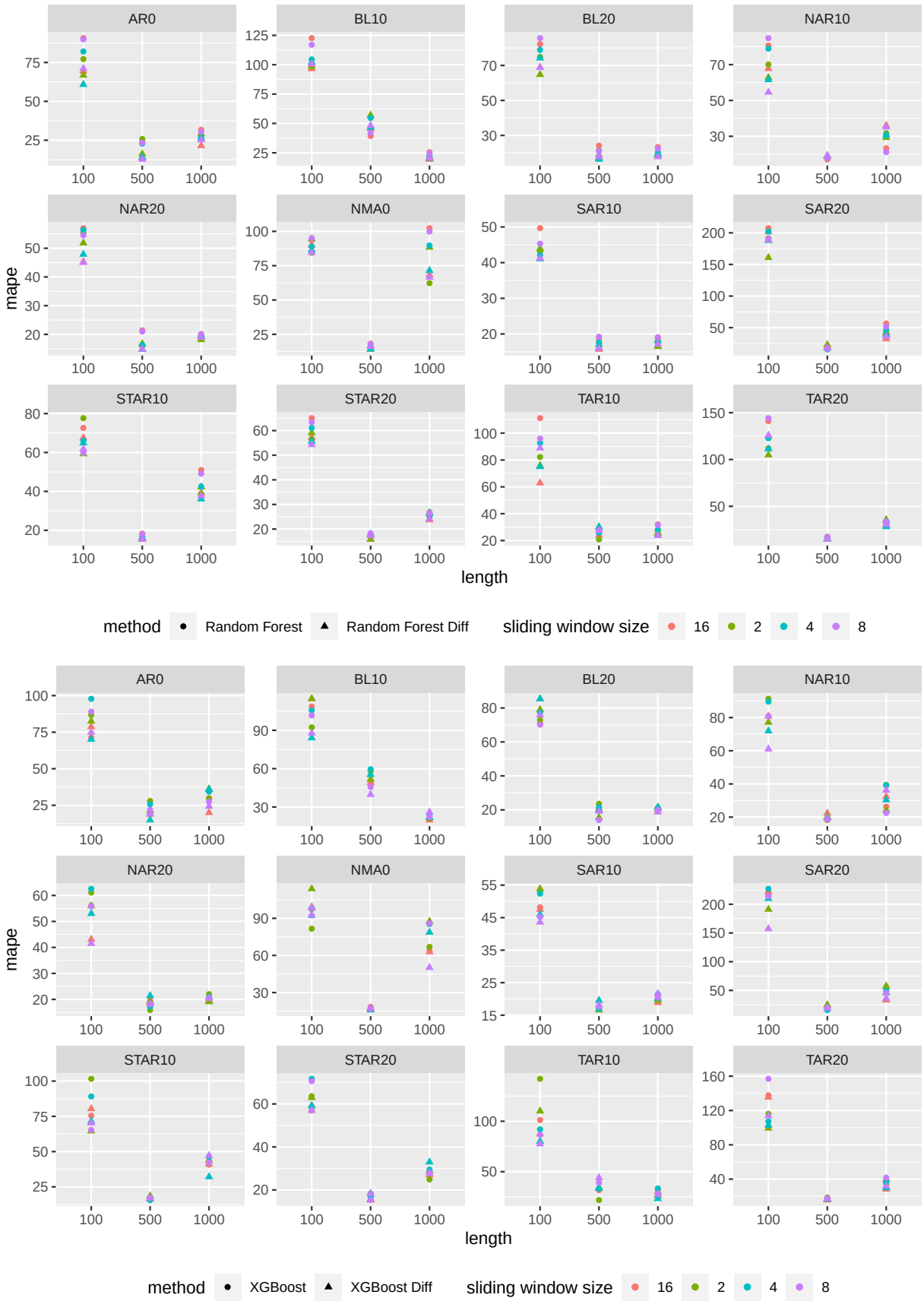

Figure A12. MAPE of the Random Forest (**above**) and XGBoost (**below**) approaches for the different data generating processes described in Table 1 superposed by a compound Poisson process.

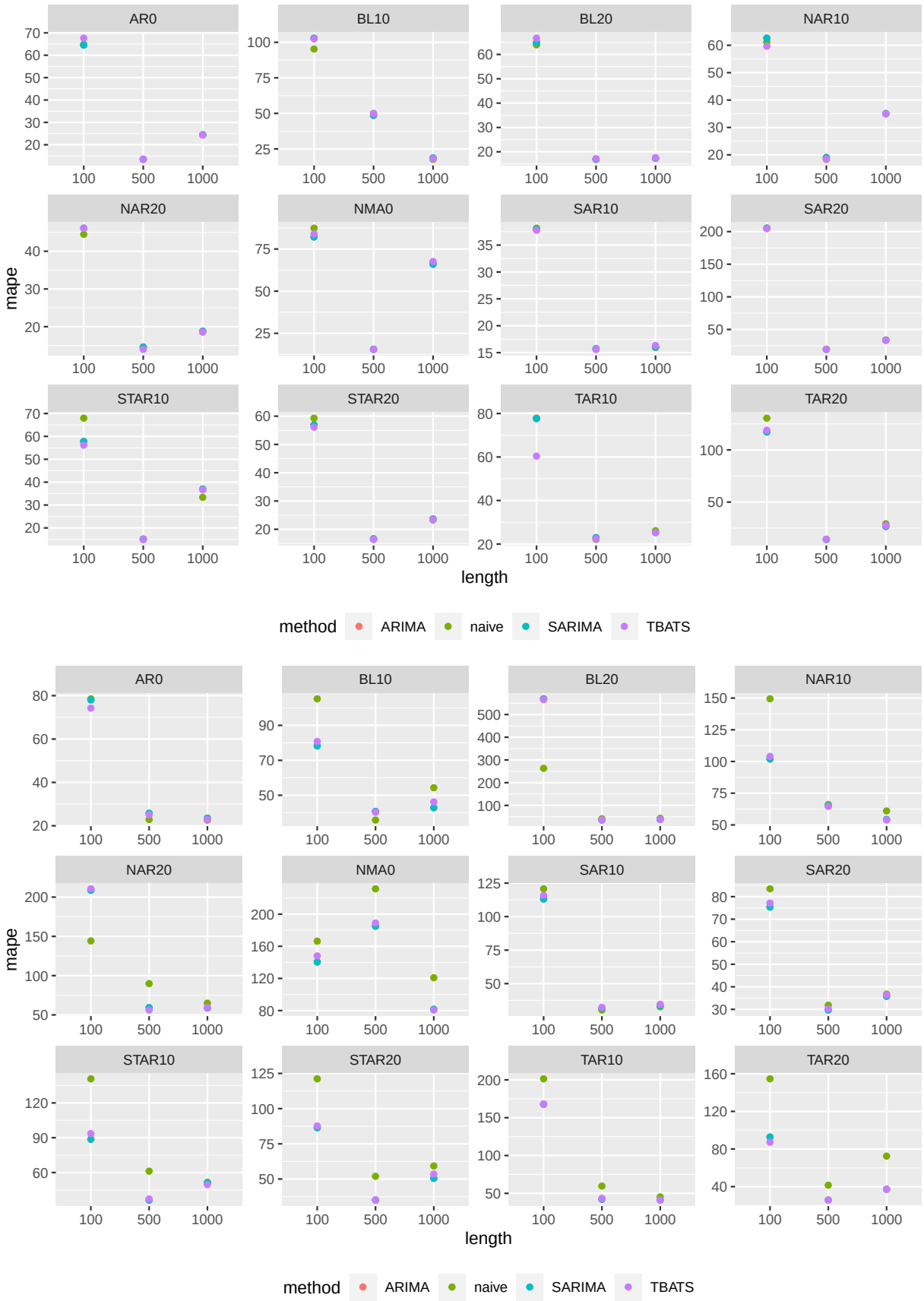

Figure A13. MAPE of the time series approaches for the different data generating processes described in Table 1 superposed by a compound Poisson process (**above**) or a random walk (**below**).

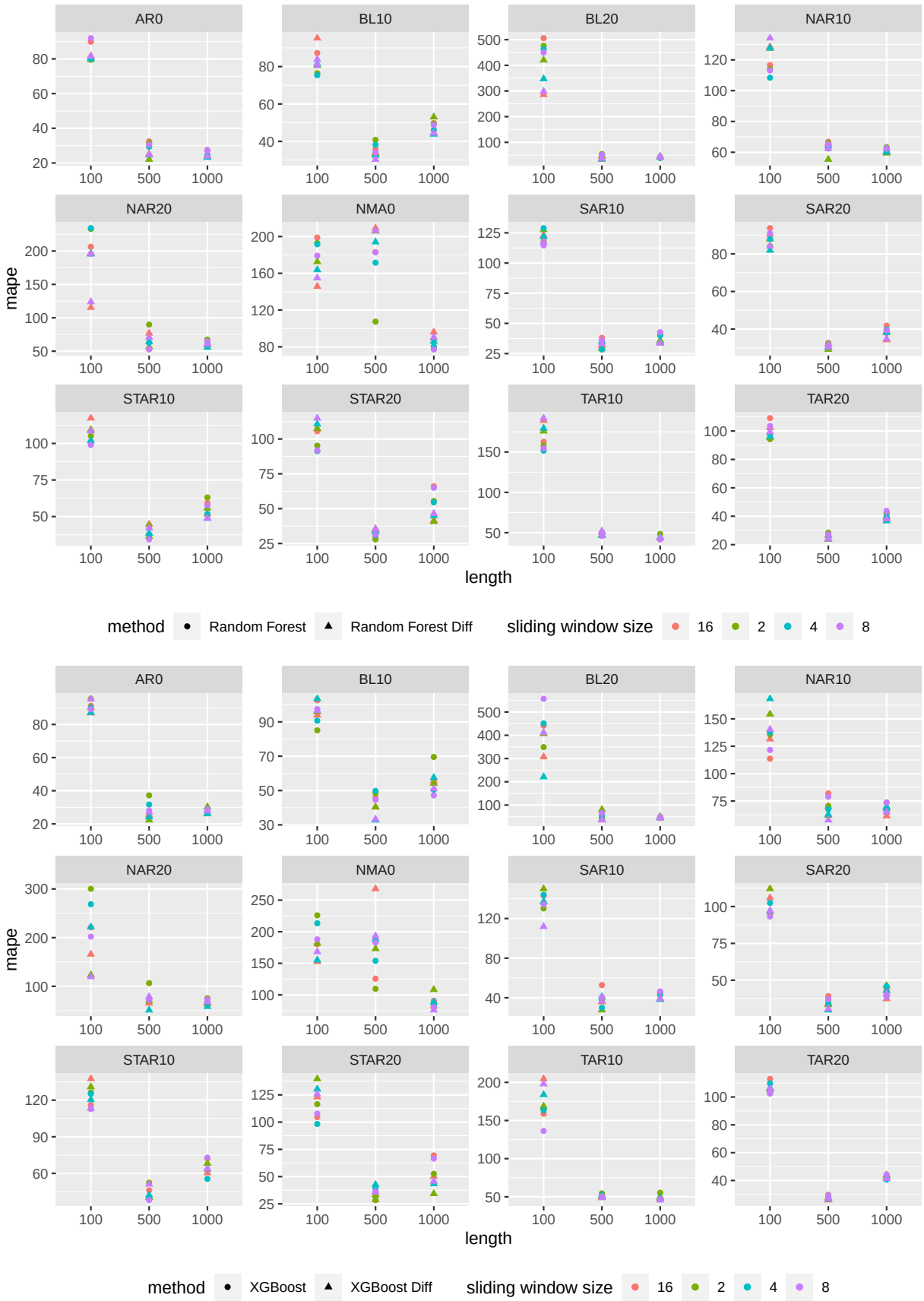

Figure A14. MAPE of the Random Forest (**above**) and XGBoost (**below**) approaches for the different data generating processes described in Table 1 superposed by a random walk.

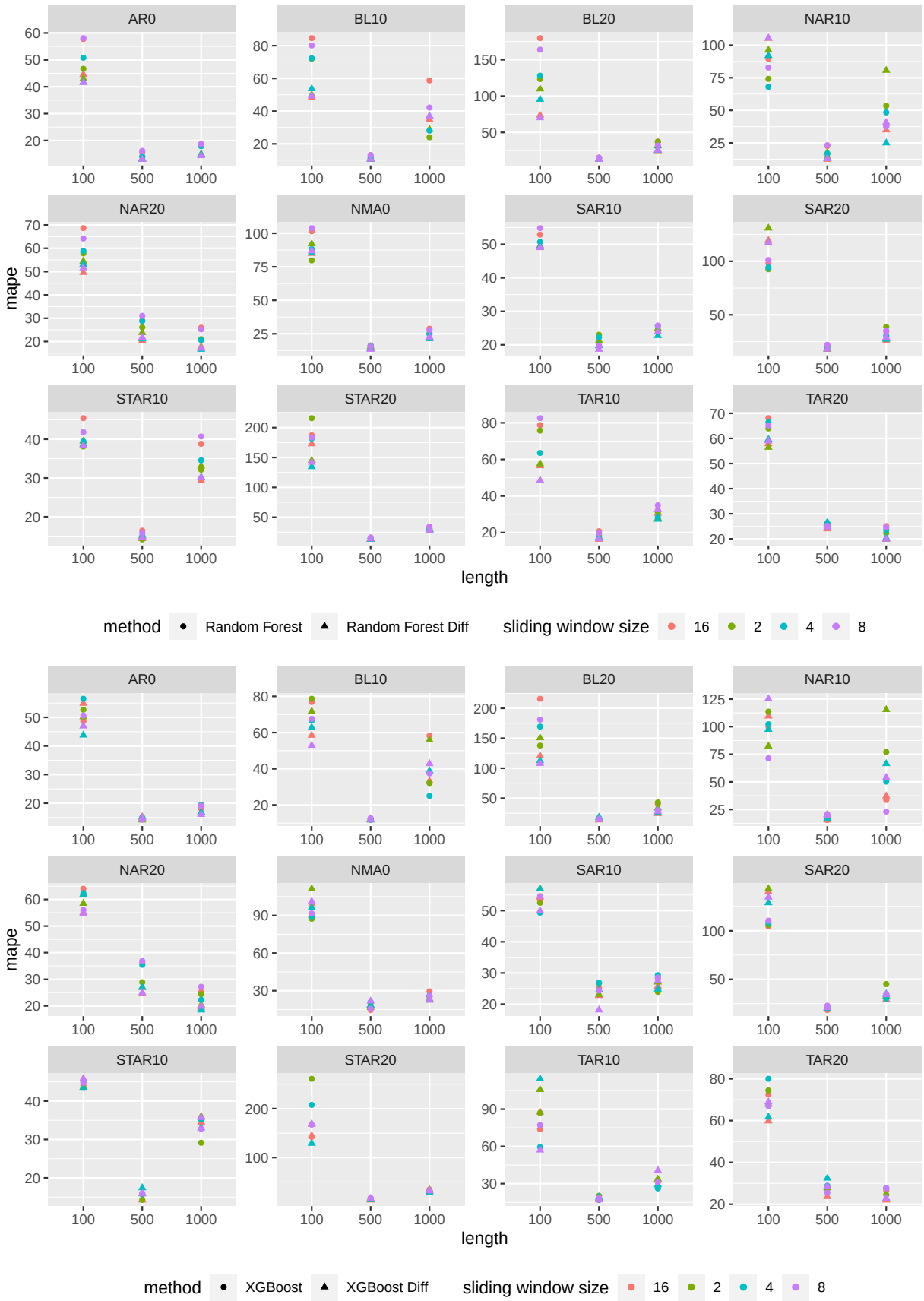

Figure A15. MAPE of the Random Forest (**above**) and XGBoost (**below**) approaches for the different data generating processes described in Table 1 superposed by a compound Poisson process and a random walk.

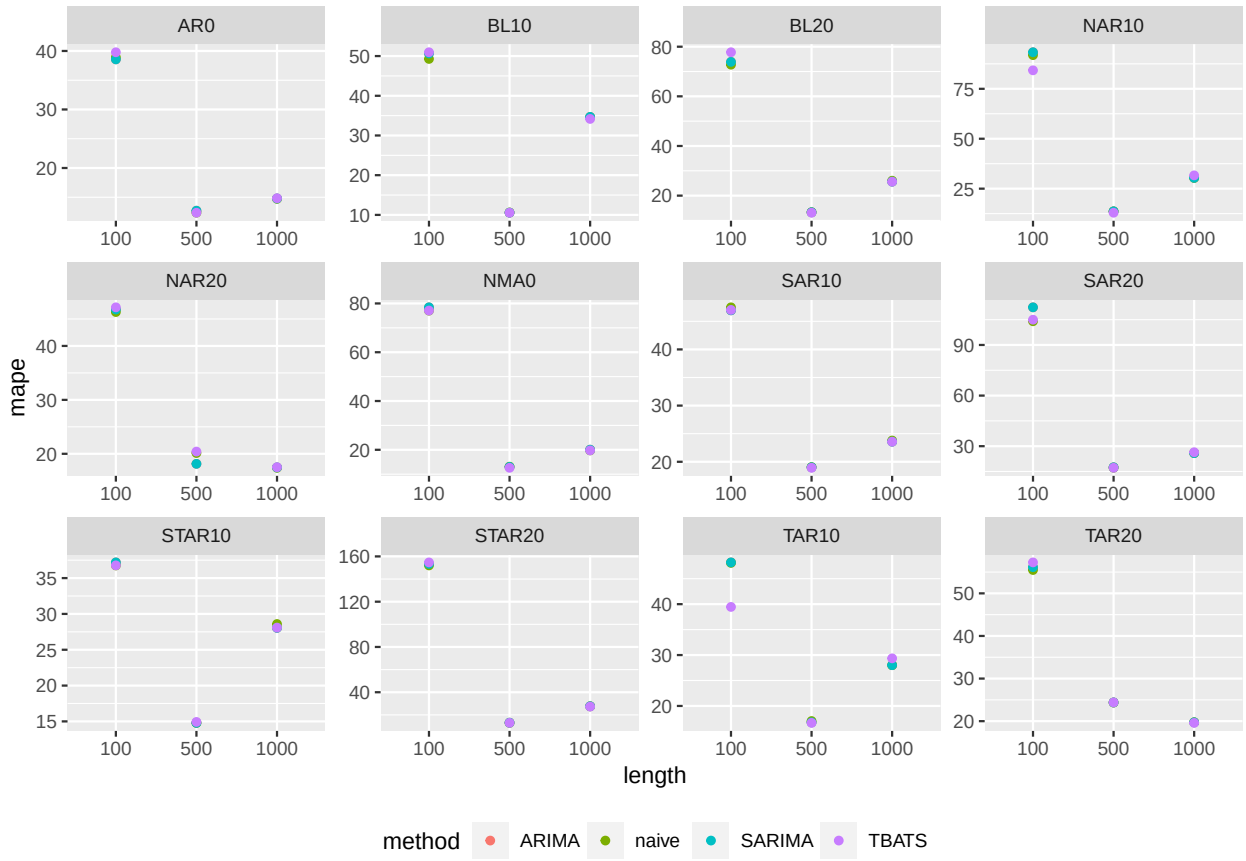

Figure A16. MAPE of the time series approaches for the different data generating processes described in Table 1 superposed by a compound Poisson process and a random walk.

References

1. Huang, H.; Pouls, M.; Meyer, A.; Pauly, M. Travel time prediction using tree-based ensembles. In Proceedings of the Computational Logistics: 11th International Conference, ICCL 2020, Enschede, The Netherlands, 28–30 September 2020; Proceedings 11; Springer: Berlin/Heidelberg, Germany, 2020; pp. 412–427.
2. Wu, C.H.; Ho, J.M.; Lee, D.T. Travel-time prediction with support vector regression. *IEEE Trans. Intell. Transp. Syst.* **2004**, *5*, 276–281. [CrossRef]
3. Lin, H.E.; Zito, R.; Taylor, M. A review of travel-time prediction in transport and logistics. *Proc. East. Asia Soc. Transp. Stud.* **2005**, *5*, 1433–1448.
4. Garrido, R.A.; Mahmassani, H.S. Forecasting freight transportation demand with the space–time multinomial probit model. *Transp. Res. Part B Methodol.* **2000**, *34*, 403–418. [CrossRef]
5. Wu, H.; Levinson, D. The ensemble approach to forecasting: A review and synthesis. *Transp. Res. Part C Emerg. Technol.* **2021**, *132*, 103357. [CrossRef]
6. Shi, Y.; Guo, X.; Yu, Y. Dynamic warehouse size planning with demand forecast and contract flexibility. *Int. J. Prod. Res.* **2018**, *56*, 1313–1325. [CrossRef]
7. Ribeiro, A.M.N.; do Carmo, P.R.X.; Endo, P.T.; Rosati, P.; Lynn, T. Short-and very short-term firm-level load forecasting for warehouses: A comparison of machine learning and deep learning models. *Energies* **2022**, *15*, 750. [CrossRef]
8. Feizabadi, J. Machine learning demand forecasting and supply chain performance. *Int. J. Logist. Res. Appl.* **2022**, *25*, 119–142. [CrossRef]
9. Kuhlmann, L.; Pauly, M. A Dynamic Systems Model for an Economic Evaluation of Sales Forecasting Methods. *Teh. Glas.* **2023**, *17*, 397–404. [CrossRef]
10. Syntetos, A.A.; Babai, Z.; Boylan, J.E.; Kolassa, S.; Nikolopoulos, K. Supply chain forecasting: Theory, practice, their gap and the future. *Eur. J. Oper. Res.* **2016**, *252*, 1–26. [CrossRef]
11. Ensafi, Y.; Amin, S.H.; Zhang, G.; Shah, B. Time-series forecasting of seasonal items sales using machine learning—A comparative analysis. *Int. J. Inf. Manag. Data Insights* **2022**, *2*, 100058. [CrossRef]

12. Wu, Z.; Ramsundar, B.; Feinberg, E.N.; Gomes, J.; Geniesse, C.; Pappu, A.S.; Leswing, K.; Pande, V. MoleculeNet: A benchmark for molecular machine learning. *Chem. Sci.* **2018**, *9*, 513–530. [CrossRef]
13. Weber, L.M.; Saelens, W.; Cannoodt, R.; Soneson, C.; Hapfelmeier, A.; Gardner, P.P.; Boulesteix, A.L.; Saeys, Y.; Robinson, M.D. Essential guidelines for computational method benchmarking. *Genome Biol.* **2019**, *20*, 125. [CrossRef] [PubMed]
14. Niemann, F.; Reining, C.; Moya Rueda, F.; Nair, N.R.; Steffens, J.A.; Fink, G.A.; Ten Hompel, M. Lara: Creating a dataset for human activity recognition in logistics using semantic attributes. *Sensors* **2020**, *20*, 4083. [CrossRef]
15. Arora, K.; Abbi, P.; Gupta, P.K. Analysis of Supply Chain Management Data Using Machine Learning Algorithms. In *Innovative Supply Chain Management via Digitalization and Artificial Intelligence*; Springer: Berlin/Heidelberg, Germany, 2022; pp. 119–133.
16. Reining, C.; Niemann, F.; Moya Rueda, F.; Fink, G.A.; ten Hompel, M. Human activity recognition for production and logistics—A systematic literature review. *Information* **2019**, *10*, 245. [CrossRef]
17. Awasthi, S.; Fernandez-Cortizas, M.; Reining, C.; Arias-Perez, P.; Luna, M.A.; Perez-Saura, D.; Roidl, M.; Gramse, N.; Klokowski, P.; Campoy, P. Micro UAV Swarm for industrial applications in indoor environment—A Systematic Literature Review. *Logist. Res.* **2023**, *16*, 1–43.
18. Friedrich, S.; Friede, T. On the role of benchmarking data sets and simulations in method comparison studies. *Biom. J.* **2023**, *66*, 2200212. [CrossRef] [PubMed]
19. Shukla, M.; Jharkharia, S. ARIMA models to forecast demand in fresh supply chains. *Int. J. Oper. Res.* **2011**, *11*, 1–18. [CrossRef]
20. Gilbert, K. An ARIMA Supply Chain Model. *Manag. Sci.* **2005**, *51*, 305–310. [CrossRef]
21. Taylor, S.J.; Letham, B. Forecasting at scale. *Am. Stat.* **2018**, *72*, 37–45. [CrossRef]
22. Kumar Jha, B.; Pande, S. Time Series Forecasting Model for Supermarket Sales using FB-Prophet. In Proceedings of the 2021 5th International Conference on Computing Methodologies and Communication (ICCMC), Erode, India, 8–10 April 2021; pp. 547–554. [CrossRef]
23. Hasmin, E.; Aini, N. Data Mining For Inventory Forecasting Using Double Exponential Smoothing Method. In Proceedings of the 2020 2nd International Conference on Cybernetics and Intelligent System (ICORIS), Manado, Indonesia, 27–28 October 2020; pp. 1–5. [CrossRef]
24. Carbonneau, R.; Laframboise, K.; Vahidov, R. Application of machine learning techniques for supply chain demand forecasting. *Eur. J. Oper. Res.* **2008**, *184*, 1140–1154. [CrossRef]
25. Wenzel, H.; Smit, D.; Sardesai, S. A literature review on machine learning in supply chain management. In *Artificial Intelligence and Digital Transformation in Supply Chain Management: Innovative Approaches for Supply Chains. Proceedings of the Hamburg International Conference of Logistics (HICL)*; epubli GmbH: Berlin, Germany, 2019; Volume 27, pp. 413–441.
26. Sharma, R.; Kamble, S.S.; Gunasekaran, A.; Kumar, V.; Kumar, A. A systematic literature review on machine learning applications for sustainable agriculture supply chain performance. *Comput. Oper. Res.* **2020**, *119*, 104926. [CrossRef]
27. Ni, D.; Xiao, Z.; Lim, M.K. A systematic review of the research trends of machine learning in supply chain management. *Int. J. Mach. Learn. Cybern.* **2020**, *11*, 1463–1482. [CrossRef]
28. Baryannis, G.; Dani, S.; Antoniou, G. Predicting supply chain risks using machine learning: The trade-off between performance and interpretability. *Future Gener. Comput. Syst.* **2019**, *101*, 993–1004. [CrossRef]
29. Kohzadi, N.; Boyd, M.S.; Kermanshahi, B.; Kaastra, I. A comparison of artificial neural network and time series models for forecasting commodity prices. *Neurocomputing* **1996**, *10*, 169–181. [CrossRef]
30. Weng, Y.; Wang, X.; Hua, J.; Wang, H.; Kang, M.; Wang, F.Y. Forecasting horticultural products price using ARIMA model and neural network based on a large-scale data set collected by web crawler. *IEEE Trans. Comput. Soc. Syst.* **2019**, *6*, 547–553. [CrossRef]
31. Siami-Namini, S.; Tavakoli, N.; Namin, A.S. A comparison of ARIMA and LSTM in forecasting time series. In Proceedings of the 2018 17th IEEE International Conference on Machine Learning and Applications (ICMLA), Orlando, FL, USA, 17–20 December 2018; IEEE: Piscataway, NJ, USA, 2018; pp. 1394–1401.
32. Palomares-Salas, J.; De La Rosa, J.; Ramiro, J.; Melgar, J.; Aguera, A.; Moreno, A. ARIMA vs. Neural networks for wind speed forecasting. In Proceedings of the 2009 IEEE International Conference on Computational Intelligence for Measurement Systems and Applications, Hong Kong, China, 11–13 May 2009; IEEE: Piscataway, NJ, USA, 2009; pp. 129–133.
33. Ampountolas, A. Modeling and forecasting daily hotel demand: A comparison based on sarimax, neural networks, and garch models. *Forecasting* **2021**, *3*, 580–595. [CrossRef]
34. Fan, D.; Sun, H.; Yao, J.; Zhang, K.; Yan, X.; Sun, Z. Well production forecasting based on ARIMA-LSTM model considering manual operations. *Energy* **2021**, *220*, 119708. [CrossRef]
35. Nyoni, T. Modeling and forecasting inflation in Kenya: Recent insights from ARIMA and GARCH analysis. *Dimorian Rev.* **2018**, *5*, 16–40.
36. Benvenuto, D.; Giovanetti, M.; Vassallo, L.; Angeletti, S.; Ciccozzi, M. Application of the ARIMA model on the COVID-2019 epidemic dataset. *Data Brief* **2020**, *29*, 105340. [CrossRef] [PubMed]
37. Tsay, R.S. Testing and modeling threshold autoregressive processes. *J. Am. Stat. Assoc.* **1989**, *84*, 231–240. [CrossRef]

38. Francq, C.; Zakoian, J.M. *GARCH Models: Structure, Statistical Inference and Financial Applications*; John Wiley & Sons: Hoboken, NJ, USA, 2019.
39. De Gooijer, J.G.; Kumar, K. Some recent developments in non-linear time series modelling, testing, and forecasting. *Int. J. Forecast.* **1992**, *8*, 135–156. [CrossRef]
40. Bontempi, G.; Ben Taieb, S.; Borgne, Y.A.L. Machine learning strategies for time series forecasting. In Proceedings of the European Business Intelligence Summer School, Brussels, Belgium, 15–21 July 2012 ; Springer: Berlin/Heidelberg, Germany, 2012; pp. 62–77.
41. Ahmed, N.K.; Atiya, A.F.; Gayar, N.E.; El-Shishiny, H. An empirical comparison of machine learning models for time series forecasting. *Econom. Rev.* **2010**, *29*, 594–621. [CrossRef]
42. Brockwell, P.J.; Davis, R.A. *Introduction to Time Series and Forecasting*; Springer: Berlin/Heidelberg, Germany, 2002.
43. Hyndman, R.J.; Athanasopoulos, G. *Forecasting: Principles and Practice*; OTexts: Melbourne, Australia, 2018.
44. Al-Saba, T.; El-Amin, I. Artificial neural networks as applied to long-term demand forecasting. *Artif. Intell. Eng.* **1999**, *13*, 189–197. [CrossRef]
45. Zhang, G.P.; Patuwo, B.E.; Hu, M.Y. A simulation study of artificial neural networks for nonlinear time-series forecasting. *Comput. Oper. Res.* **2001**, *28*, 381–396. [CrossRef]
46. Hwarng, H.B. Insights into neural-network forecasting of time series corresponding to ARMA (p, q) structures. *Omega* **2001**, *29*, 273–289. [CrossRef]
47. Box, G.E.; Jenkins, G.M.; Reinsel, G.C.; Ljung, G.M. *Time Series Analysis: Forecasting and Control*; John Wiley & Sons: Hoboken, NJ, USA, 2015.
48. Shumway, R.H.; Stoffer, D.S.; Stoffer, D.S. *Time Series Analysis and Its Applications*; Springer: Berlin/Heidelberg, Germany, 2000.
49. De Livera, A.M.; Hyndman, R.J.; Snyder, R.D. Forecasting time series with complex seasonal patterns using exponential smoothing. *J. Am. Stat. Assoc.* **2011**, *106*, 1513–1527. [CrossRef]
50. Box, G.E.; Cox, D.R. An Analysis of Transformations. *J. R. Stat. Soc. Ser. B (Methodol.)* **1964**, *26*, 211–243. [CrossRef]
51. Ji, S.; Wang, X.; Zhao, W.; Guo, D. An application of a three-stage XGBoost-based model to sales forecasting of a cross-border e-commerce enterprise. *Math. Probl. Eng.* **2019**, *2019*, 8503252. [CrossRef]
52. Islam, S.; Amin, S.H. Prediction of probable backorder scenarios in the supply chain using Distributed Random Forest and Gradient Boosting Machine learning techniques. *J. Big Data* **2020**, *7*, 65. [CrossRef]
53. Ma, Y.; Zhang, Z.; Ihler, A.; Pan, B. Estimating warehouse rental price using machine learning techniques. *Int. J. Comput. Commun. Control* **2018**, *13*, 235–250. [CrossRef]
54. Kuhlmann, L.; Wilmes, D.; Müller, E.; Pauly, M.; Horn, D. RODD: Robust Outlier Detection in Data Cubes. *arXiv* **2023**, arXiv:2303.08193.
55. Aguilar Madrid, E.; Antonio, N. Short-term electricity load forecasting with machine learning. *Information* **2021**, *12*, 50. [CrossRef]
56. Chen, T.; Guestrin, C. XGBoost: A Scalable Tree Boosting System. In Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. Association for Computing Machinery, San Francisco, CA, USA, 13–17 August 2016; pp. 785–794.
57. Therneau, T.M.; Atkinson, E.J. *An Introduction to Recursive Partitioning Using the RPART Routines*; Technical Report; Mayo Foundation: Rochester, MN, USA, 1997.
58. Schapire, R.E.; Freund, Y. Boosting: Foundations and algorithms. *Kybernetes* **2013**, *42*, 164–166. [CrossRef]
59. Friedman, J.H. Stochastic gradient boosting. *Comput. Stat. Data Anal.* **2002**, *38*, 367–378. [CrossRef]
60. Mayr, A.; Binder, H.; Gefeller, O.; Schmid, M. The evolution of boosting algorithms. *Methods Inf. Med.* **2014**, *53*, 419–427.
61. Morde, V. XGBoost Algorithm: Long May She Reign! Available online: https://towardsdatascience.com/https-medium-com-vishalmorde-xgboost-algorithm-long-she-may-rein-edd9f99be63d (accessed on 13 December 2023).
62. Luo, J.; Zhang, Z.; Fu, Y.; Rao, F. Time series prediction of COVID-19 transmission in America using LSTM and XGBoost algorithms. *Results Phys.* **2021**, *27*, 104462. [CrossRef]
63. Alim, M.; Ye, G.H.; Guan, P.; Huang, D.S.; Zhou, B.S.; Wu, W. Comparison of ARIMA model and XGBoost model for prediction of human brucellosis in mainland China: A time-series study. *BMJ Open* **2020**, *10*, e039676. [CrossRef]
64. Zhang, L.; Bian, W.; Qu, W.; Tuo, L.; Wang, Y. Time series forecast of sales volume based on XGBoost. In *Proceedings of the Journal of Physics: Conference Series*; IOP Publishing: Bristol, UK, 2021; Volume 1873, p. 012067.
65. Breiman, L. Random Forests. *Mach. Learn.* **2001**, *45*, 5–32. [CrossRef]
66. Breiman, L.; Friedman, J.H.; Olshen, R.A.; Stone, C.J. *Classification and Regression Trees*; Routledge: London, UK, 2017.
67. Wright, M.N.; Ziegler, A. ranger: A Fast Implementation of Random Forests for High Dimensional Data in C++ and R. *J. Stat. Softw.* **2017**, *77*, 1–17. [CrossRef]
68. Geurts, P.; Ernst, D.; Wehenkel, L. Extremely randomized trees. *Mach. Learn.* **2006**, *63*, 3–42. [CrossRef]
69. Goehry, B.; Yan, H.; Goude, Y.; Massart, P.; Poggi, J.M. Random Forests for Time Series. *REVSTAT-Stat. J.* **2023**, *21*, 283–302.
70. Pórtoles, J.; González, C.; Moguerza, J.M. Electricity price forecasting with dynamic trees: A benchmark against the random forest approach. *Energies* **2018**, *11*, 1588. [CrossRef]

71. Kane, M.J.; Price, N.; Scotch, M.; Rabinowitz, P. Comparison of ARIMA and Random Forest time series models for prediction of avian influenza H5N1 outbreaks. *BMC Bioinform.* **2014**, *15*, 276. [CrossRef]
72. Salari, N.; Liu, S.; Shen, Z.J.M. Real-time delivery time forecasting and promising in online retailing: When will your package arrive? *Manuf. Serv. Oper. Manag.* **2022**, *24*, 1421–1436. [CrossRef]
73. Vairagade, N.; Logofatu, D.; Leon, F.; Muharemi, F. Demand forecasting using random forest and artificial neural network for supply chain management. In Proceedings of the Computational Collective Intelligence: 11th International Conference, ICCCI 2019, Hendaye, France, 4–6 September 2019; Proceedings, Part I 11; Springer: Berlin/Heidelberg, Germany, 2019; pp. 328–339.
74. R Core Team. *R: A Language and Environment for Statistical Computing*; R Foundation for Statistical Computing: Vienna, Austria, 2022.
75. Hyndman, R.J.; Khandakar, Y. Automatic time series forecasting: The forecast package for R. *J. Stat. Softw.* **2008**, *26*, 1–22. [CrossRef]
76. Chen, T.; He, T.; Benesty, M.; Khotilovich, V.; Tang, Y.; Cho, H.; Chen, K.; Mitchell, R.; Cano, I.; Zhou, T.; et al. xgboost: Extreme Gradient Boosting; R Package Version 1.6.0.1. Available online: https://CRAN.R-project.org/package=xgboost (accessed on 10 July 2024).
77. Luong, H.T. Measure of bullwhip effect in supply chains with autoregressive demand process. *Eur. J. Oper. Res.* **2007**, *180*, 1086–1097. [CrossRef]
78. Ivanov, D.; Dolgui, A. Viability of intertwined supply networks: Extending the supply chain resilience angles towards survivability. A position paper motivated by COVID-19 outbreak. *Int. J. Prod. Res.* **2020**, *58*, 2904–2915. [CrossRef]
79. Kingman, J.F.C. *Poisson Processes*; Clarendon Press: Oxford, UK, 1992; Volume 3.
80. Sheffi, Y. *The Resilient Enterprise: Overcoming Vulnerability for Competitive Advantage*; MIT Press: Cambridge, MA, USA, 2005.
81. Chatfield, C.; Xing, H. *The Analysis of Time Series: An Introduction with R*; Chapman and hall/CRC: Boca Raton, FL, USA, 2019.
82. Box, G. Signal-to-noise ratios, performance criteria, and transformations. *Technometrics* **1988**, *30*, 1–17. [CrossRef]
83. Cooper, R.B. Queueing theory. In *Proceedings of the ACM'81 Conference*; Association for Computing Machinery: New York, NY, USA, 1981; pp. 119–122.
84. Artalejo, J.R.; Lopez-Herrero, M. Analysis of the busy period for the M/M/c queue: An algorithmic approach. *J. Appl. Probab.* **2001**, *38*, 209–222. [CrossRef]
85. Schwarz, M.; Sauer, C.; Daduna, H.; Kulik, R.; Szekli, R. M/M/1 queueing systems with inventory. *Queueing Syst.* **2006**, *54*, 55–78. [CrossRef]
86. Kobayashi, H.; Konheim, A. Queueing models for computer communications system analysis. *IEEE Trans. Commun.* **1977**, *25*, 2–29. [CrossRef]
87. Gautam, N. *Analysis of Queues: Methods and Applications*; CRC Press: Boca Raton, FL, USA, 2012.
88. Brown, L.; Gans, N.; Mandelbaum, A.; Sakov, A.; Shen, H.; Zeltyn, S.; Zhao, L. Statistical analysis of a telephone call center: A queueing-science perspective. *J. Am. Stat. Assoc.* **2005**, *100*, 36–50. [CrossRef]
89. Green, L. Queueing analysis in healthcare. In *Patient Flow: Reducing Delay in Healthcare Delivery*; Springer: Berlin/Heidelberg, Germany, 2006; pp. 281–307.
90. Radmilovic, Z.; Colic, V.; Hrle, Z. Some aspects of storage and bulk queueing systems in transport operations. *Transp. Plan. Technol.* **1996**, *20*, 67–81. [CrossRef]
91. Dietterich, T.G. Machine learning for sequential data: A review. In Proceedings of the Structural, Syntactic, and Statistical Pattern Recognition: Joint IAPR International Workshops SSPR 2002 and SPR 2002, Windsor, ON, Canada, 6–9 August 2002; Proceedings; Springer: Berlin/Heidelberg, Germany, 2002; pp. 15–30.
92. Savva, A.D.; Kassinopoulos, M.; Smyrnis, N.; Matsopoulos, G.K.; Mitsis, G.D. Effects of motion related outliers in dynamic functional connectivity using the sliding window method. *J. Neurosci. Methods* **2020**, *330*, 108519. [CrossRef] [PubMed]
93. Ferreira, R.; Martiniano, A.; Ferreira, A.; Ferreira, A.; Sassi, R. Daily Demand Forecasting Orders. UCI Machine Learning Repository. 2017. Available online: https://archive.ics.uci.edu/dataset/409/daily+demand+forecasting+orders (accessed on 10 June 2024).
94. Dua, D.; Graff, C. UCI Machine Learning Repository. 2017. Available online: http://archive.ics.uci.edu/datasets (accessed on 10 June 2024).
95. Probst, P.; Boulesteix, A.L.; Bischl, B. Tunability: Importance of Hyperparameters of Machine Learning Algorithms. *J. Mach. Learn. Res.* **2019**, *20*, 1–32.
96. Fernández-Delgado, M.; Cernadas, E.; Barro, S.; Amorim, D. Do we Need Hundreds of Classifiers to Solve Real World Classification Problems? *J. Mach. Learn. Res.* **2014**, *15*, 3133–3181.
97. Venkatapathy, A.K.R.; Riesner, A.; Roidl, M.; Emmerich, J.; ten Hompel, M. PhyNode: An intelligent, cyber-physical system with energy neutral operation for PhyNetLab. In Proceedings of the Smart SysTech 2015; European Conference on Smart Objects, Systems and Technologies, VDE, Aachen, Germany, 16–17 July 2015; pp. 1–8.

98. Gouda, A.; Heinrich, D.; Hünnefeld, M.; Priyanta, I.F.; Reining, C.; Roidl, M. A Grid-based Sensor Floor Platform for Robot Localization using Machine Learning. In Proceedings of the 2023 IEEE International Instrumentation and Measurement Technology Conference (I2MTC), Kuala Lumpur, Malaysia, 22–25 May 2023; IEEE: Piscataway, NJ, USA, 2023; pp. 1–6.
99. Aladag, C.H.; Egrioglu, E.; Kadilar, C. Forecasting nonlinear time series with a hybrid methodology. *Appl. Math. Lett.* **2009**, *22*, 1467–1470. [CrossRef]
100. Zhang, G.P. Time series forecasting using a hybrid ARIMA and neural network model. *Neurocomputing* **2003**, *50*, 159–175. [CrossRef]
101. Smyl, S. A hybrid method of exponential smoothing and recurrent neural networks for time series forecasting. *Int. J. Forecast.* **2020**, *36*, 75–85. [CrossRef]

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.